

UNIVERSIDAD PRIVADA DOMINGO SAVIO



LICENCIATURA EN INGENIERÍA EN SISTEMAS

"IMPLEMENTACION DE UN SISTEMA WEB PARA LA GESTIÓN BANCARIA CON OPERACIONES FINANCIERAS"

PROYECTO SOCIOFORMATIVO

NOMBRE DEL ESTUDIANTE

Luis Mario Rocha Vela

Douglas Flor Hernández

DOCENTE

Macias Burgos Miguel Ángel

Santa Cruz de la Sierra – Bolivia

2025

INDICE

INTRODUCCIÓN	4
DEDICATORIA	5
AGRADECIMIENTOS	5
CAPITULO I	6
ANTECEDENTES	6
1.1 PLANTEAMIENTO DEL PROBLEMA	6
1.2 JUSTIFICACION	6
1.3 TIPO Y NIVEL DE INVESTIGACION	7
1.4 OBJETIVOS DEL PROYECTO	7
1.4.1 Objetivo General	7
1.4.2 Objetivos específicos	7
CAPITULO II	8
METODOLOGÍA DEL TRABAJO	8
2.1 ORGANIGRAMA DE LA ORGANIZACIÓN GRUPAL	8
2.2 DESCRIPCIÓN DEL ÁMBITO DE LA INVESTIGACIÓN	9
CAPITULO III	10
MARCO CONCEPTUAL	10
3.1 VISUAL STUDIO CODE (VSCODE)	10
3.2 HYPERTEXT MARKUP LANGUAGE	11
3.3 CASCADING STYLE SHEETS	12
3.4 PHP: Lenguaje de Programación para Desarrollo Web	13
3.5 MySQL: Sistema de Gestión de Bases de Datos Relacionales	14
3.6 XAMPP: Entorno de Desarrollo para Servidores Web	15

CAPITULO IV	16
MODELADO DE LOS ARTEFACTOS DEL SISTEMA	16
4.1 LISTA DE REQUERIMIENTOS.....	17
4.2 MODELADO ESTRUCTURAL	19
4.2.1 Diagrama de clases.....	19
4.2.2 Diagrama de objetos.....	20
4.2.3 Diagrama de paquetes.....	22
4.3 MODELADO DE COMPORTAMIENTO	23
4.3.1 Diagrama de casos de uso	23
4.3.2 Diagrama de secuencia	26
CAPITULO V.....	28
DESARROLLO DE LA SISTEMA FUNCIONAL	28
CAPITULO VI	53
CONCLUSIONES DEL PROYECTO.....	53
CAPITULO VII.....	54
RECOMENDACIONES GRUPALES	54
CAPITULO VIII	55
REFERENCIAS BIBLIOGRAFICAS.....	55

INTRODUCCIÓN

El desarrollo de sistemas bancarios ha evolucionado significativamente en respuesta a las crecientes necesidades de seguridad, eficiencia y accesibilidad en los servicios financieros. Desde los primeros registros manuales hasta las plataformas digitales actuales, la tecnología ha permitido optimizar los procesos bancarios, facilitando la gestión de cuentas y transacciones de manera más ágil y segura. En este contexto, el presente proyecto tiene como objetivo la construcción de un sistema web modular para la gestión bancaria, enfocado en proporcionar soporte a los cajeros automáticos (ATM) mediante un servicio web basado en API REST.

El módulo bancario desarrollado permitirá realizar operaciones financieras clave, como la apertura de cuentas para personas naturales, depósitos y retiros de fondos, consulta de saldo y generación de informes de transacciones en función de un rango de fechas. Además, el sistema contará con funcionalidades complementarias, como la compatibilidad con múltiples idiomas y la posibilidad de implementar una Aplicación Web Progresiva (PWA) para mejorar la experiencia del usuario.

Desde el punto de vista tecnológico, el sistema se implementará utilizando PHP como lenguaje de desarrollo en el backend y MySQL como sistema de gestión de bases de datos, bajo una arquitectura Modelo-Vista-Controlador (MVC). El frontend se desarrollará con HTML5 y CSS, garantizando una interfaz intuitiva y accesible. Asimismo, el sistema quedará alojado en un servidor en la nube, específicamente en Cloud Server de 4Code, asegurando su disponibilidad y escalabilidad.

Para garantizar un diseño estructurado y bien documentado, se empleará el enfoque UML (Unified Modeling Language), a incluir el modelado estructural a través del diagrama de clases y el modelado de comportamiento mediante diagramas de casos de uso y diagramas de secuencia. Entre los diagramas clave se incluyen:

- Caso de Uso Principal del Sistema
- Caso de Uso para la Gestión de Cuentas Bancarias
- Caso de Uso para la Apertura de Nuevas Cuentas
- Diagrama de Secuencia para las operaciones de depósito y retiro

Este proyecto busca no solo optimizar la gestión de operaciones bancarias mediante una solución escalable y eficiente, sino también demostrar la aplicación de principios de programación orientada a objetos, diseño estructural y desarrollo de servicios web. Su implementación permitirá mejorar la accesibilidad a los servicios financieros, optimizar la administración de cuentas y ofrecer una alternativa digital segura para el manejo de transacciones bancarias.

DEDICATORIA

Este proyecto está dedicado al Ingeniero Miguel Ángel Macías Burgos, docente de la materia Desarrollo de Sistemas I, por su compromiso, orientación y apoyo constante durante todo el proceso de desarrollo del sistema bancario. Su conocimiento y experiencia han sido fundamentales para guiarnos en cada etapa del diseño, modelado y programación del sistema, permitiéndonos fortalecer nuestras habilidades técnicas y nuestra comprensión de los principios del desarrollo. Gracias a su dedicación y valiosas enseñanzas, hemos logrado enfrentar y superar los desafíos del proyecto, consolidando una base sólida para nuestro crecimiento profesional.

AGRADECIMIENTOS

Expresamos nuestro más sincero agradecimiento al Ingeniero Miguel Ángel Macías Burgos, por su paciencia, guía y exigencia académica, que nos motivaron a esforzarnos al máximo en cada aspecto del desarrollo del sistema. Su apoyo y retroalimentación han sido clave para garantizar la calidad y funcionalidad del proyecto.

A nuestras familias, por su apoyo incondicional, comprensión y motivación a lo largo de todo el proceso. Su respaldo nos permitió afrontar los desafíos con determinación y perseverancia.

A nuestros compañeros de equipo, por su compromiso, esfuerzo y colaboración en cada fase del proyecto. La cooperación y el trabajo en equipo han sido esenciales para alcanzar los objetivos planteados y culminar con éxito este desarrollo.

Finalmente, agradecemos a todas aquellas personas que, de manera directa o indirecta, contribuyeron con sus conocimientos, sugerencias y apoyo técnico, ayudándonos a mejorar y perfeccionar este sistema bancario, asegurando su solidez y aplicabilidad en el ámbito real.

ANTECEDENTES

Este capítulo presenta el contexto del proyecto, exponiendo los antecedentes, el problema identificado y la justificación de su desarrollo. Se analiza la necesidad de implementar un sistema bancario digital que optimice la gestión de cuentas y transacciones, mejorando la eficiencia y seguridad de los procesos financieros.

1.1 PLANTEAMIENTO DEL PROBLEMA

El sector bancario requiere sistemas robustos y seguros para la gestión eficiente de sus operaciones financieras. Aunque existen plataformas centralizadas que integran cajeros automáticos (ATM) con los sistemas bancarios, su desarrollo e implementación suelen estar limitados a grandes instituciones, dejando pocas oportunidades para el aprendizaje y la experimentación académica.

Ante esta realidad, se propone el desarrollo de un sistema propio de banca digital que incluya un módulo API para la integración con ATM. A diferencia de las soluciones comerciales, este proyecto se enfocará en la creación de una plataforma con código estructurado y documentación detallada, permitiendo a estudiantes y desarrolladores comprender su funcionamiento y replicarlo en distintos entornos. Además, se garantizará el cumplimiento de estándares de seguridad y eficiencia en la gestión de transacciones financieras, asegurando una solución viable tanto a nivel académico como en su posible aplicación en escenarios reales.

1.2 JUSTIFICACION

Este proyecto se justifica desde una doble perspectiva: técnica y académica. Desde el punto de vista técnico, busca desarrollar un sistema bancario propio que optimice la administración de cuentas y transacciones mediante un módulo API seguro y escalable. Esto permitirá la automatización de procesos como depósitos, retiros y transferencias en tiempo real, reduciendo errores y mejorando la disponibilidad del servicio.

Desde la perspectiva académica, este desarrollo representa una oportunidad para aplicar conocimientos en modelado de software, arquitectura de sistemas, seguridad informática y protocolos de comunicación. A diferencia de las soluciones comerciales cerradas, este sistema estará acompañado de documentación detallada, facilitando su comprensión y reutilización por parte de otros desarrolladores.

1.3 TIPO Y NIVEL DE INVESTIGACION

La investigación se divide en dos enfoques: teórico, que abarcó el estudio de principios y tecnologías para el sistema bancario, y experimental, que consistió en la implementación y prueba práctica del sistema.

- **Descriptivo:** Se exploraron las tecnologías clave para el desarrollo del sistema bancario, como la arquitectura de sistemas, la seguridad y la integración con cajeros automáticos. Se enfocó en aspectos como la gestión de transacciones, la creación de un sistema modular escalable y accesible, y las medidas necesarias para asegurar la protección de las operaciones financieras.
- **Experimental:** Basado en la investigación teórica, se desarrolló el sistema utilizando herramientas como PHP y MySQL. Se implementó un módulo API REST y se diseñó una interfaz para el sistema; utilizando herramientas como HTML y CSS, realizando pruebas de integración para garantizar su funcionamiento correcto y optimizando la seguridad de los datos sensibles.

1.4 OBJETIVOS DEL PROYECTO

En esta sección se detallan los objetivos que guían el desarrollo del proyecto. Se presenta el objetivo general que define el propósito principal del proyecto, seguido de los objetivos específicos que abordan las metas concretas y las acciones necesarias para alcanzar el resultado esperado.

1.4.1 Objetivo General

Desarrollar un módulo de servicio API para la integración de cajeros automáticos (ATM) con el sistema bancario, garantizando la seguridad, eficiencia y disponibilidad en la gestión de transacciones financieras en tiempo real.

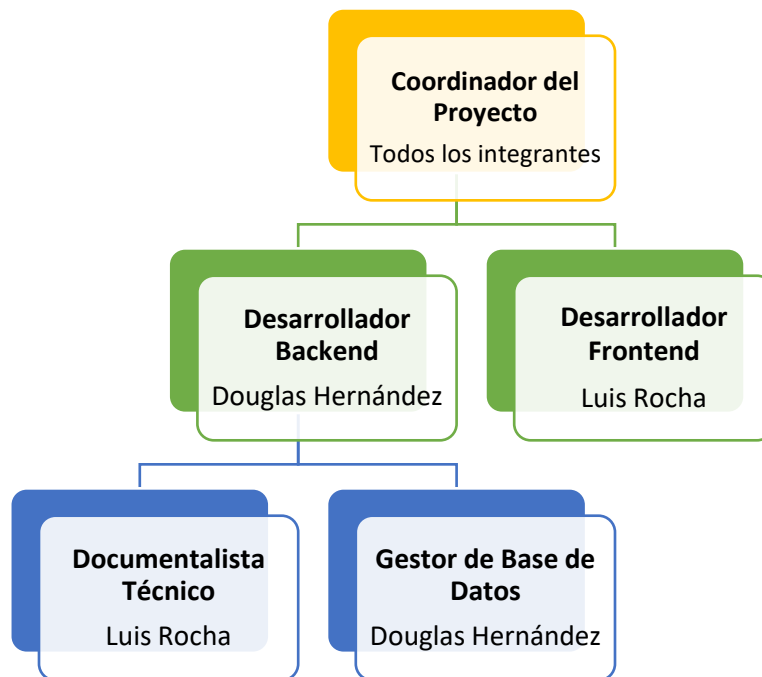
1.4.2 Objetivos específicos

- Modelar la estructura y el comportamiento del sistema mediante UML, utilizando diagramas de clases, casos de uso, secuencia y actividad, para definir de manera clara y detallada la interacción entre todos los módulos.
- Desarrollar una plataforma segura y eficiente que permita la apertura y administración de cuentas bancarias, gestión de usuarios, autenticación robusta y ejecución de transacciones como depósitos, retiros y transferencias.
- Implementar un módulo API para la integración con cajeros automáticos (ATM), estableciendo endpoints y protocolos de comunicación para garantizar transacciones en tiempo real.
- Garantizar la seguridad y eficiencia del sistema, aplicando buenas prácticas en el desarrollo, gestión de datos y la implementación de funcionalidades que mejoraran la experiencia del usuario.

METODOLOGÍA DEL TRABAJO

Este capítulo presenta la organización del grupo mediante un organigrama visual, detallando la estructura interna de coordinación. Además, describe los entornos claves, como el laboratorio de informática, la biblioteca universitaria y la casa epicentro, que fueron fundamentales para la planificación, el desarrollo y la culminación exitosa del trabajo.

2.1 ORGANIGRAMA DE LA ORGANIZACIÓN GRUPAL



En la presentación del organigrama de posición organizacional, enfatizamos el importante papel de cada miembro del equipo, reconociendo sus aportes específicos en áreas clave del proyecto. Este enfoque permite una visualización clara de cómo cada participante contribuyó con sus habilidades y conocimientos para el logro exitoso de los objetivos de la investigación. Por otro lado, la transparencia y el detalle no solo resaltan la diversidad de talentos en el equipo, sino que también resaltan la colaboración efectiva que fue clave para el éxito general del proyecto.

2.2 DESCRIPCIÓN DEL ÁMBITO DE LA INVESTIGACIÓN

Durante el desarrollo del proyecto, se identificaron tres entornos clave que fueron fundamentales para su implementación y éxito:

- **Laboratorio de Informática:** Este espacio fue crucial para la planificación y organización del proyecto. Aquí se definieron las tareas del cronograma y se llevaron a cabo reuniones para coordinar el diseño y la implementación del módulo API para la integración de cajeros automáticos (ATM) con el sistema bancario. En este entorno se modeló la estructura y el comportamiento del sistema utilizando UML, incluyendo diagramas de clases, casos de uso, secuencia y actividad. Además, se establecieron las estrategias para la implementación de la base de datos en MySQL, la seguridad de la API y la comunicación entre los cajeros automáticos y el sistema bancario.
- **Biblioteca de la Universidad:** Este entorno fue esencial para la investigación teórica y el análisis de normativas bancarias aplicables al desarrollo del módulo API. En este espacio se revisaron estándares de seguridad, protocolos de comunicación y metodologías para garantizar la autenticación segura de usuarios y la integridad de las transacciones financieras. También se realizó la documentación de pruebas de rendimiento y seguridad, asegurando el cumplimiento de estándares bancarios en la gestión de depósitos, retiros y transferencias.
- **Espacio de Trabajo Remoto:** El hogar de uno de los integrantes del equipo se convirtió en un entorno clave para la integración y pruebas finales del sistema. Aquí se llevaron a cabo sesiones de revisión y depuración del código, pruebas de funcionalidad en entornos simulados y ajustes en la comunicación entre la API y los cajeros automáticos. Además, fue el espacio donde el equipo se preparó para la presentación del proyecto, asegurando que la documentación y el sistema estuvieran en condiciones óptimas para su evaluación.

Estos tres entornos fueron fundamentales para el desarrollo e implementación del módulo API, proporcionando los espacios necesarios para garantizar la seguridad, estabilidad y eficiencia del sistema en el procesamiento de transacciones bancarias en tiempo real.

MARCO CONCEPTUAL

Este capítulo proporciona las bases teóricas y conceptuales necesarias para comprender los elementos clave del proyecto, definiendo los términos fundamentales y estableciendo un marco de referencia que facilita el análisis y la aplicación de la información.

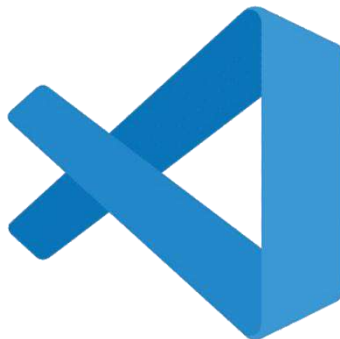
3.1 VISUAL STUDIO CODE (VSCODE)

Visual Studio Code, lanzado en 2015 por Microsoft, se ha convertido en uno de los editores de código más utilizados debido a su ligereza, flexibilidad y la enorme comunidad que lo respalda.

VSCoode es un editor de código fuente gratuito y abierto que proporciona soporte para una amplia variedad de lenguajes de programación. Ofrece una interfaz simple pero poderosa para desarrollar sitios web, y su enfoque en la personalización lo hace ideal para proyectos individuales.

Características clave:

- **Extensiones y plugins:** VSCoode permite la instalación de extensiones para lenguajes específicos, herramientas de depuración, integración con bases de datos, y más.
- **Integración con Git:** Permite gestionar versiones de código de manera eficiente, facilitando el seguimiento de los cambios en el proyecto.
- **Personalización de la interfaz:** Los usuarios pueden ajustar la apariencia y funcionalidad del editor según sus necesidades. (Cuadrado, 2022) (recluit, 2022)



3.2 HYPERTEXT MARKUP LANGUAGE

Fue creado por Tim Berners-Lee en 1991 como el lenguaje de marcado estándar para estructurar contenido en la web.

HTML es el lenguaje de marcado utilizado para estructurar páginas web. Permite definir la jerarquía y los elementos dentro de una página, como encabezados, párrafos, enlaces y tablas.

Características clave:

- **Estructuración del contenido:** Utiliza etiquetas para definir elementos como texto, imágenes, formularios, y tablas.
- **Enlaces y navegación:** Permite crear vínculos a otras páginas o sitios web.
- **Multimedia:** Soporta la inserción de contenido multimedia como imágenes, videos y audios.
- **Uso en el proyecto:** HTML fue utilizado para estructurar todo el contenido del sitio web, desde los encabezados y párrafos hasta las tablas y formularios necesarios para la interacción del usuario.

En este proyecto, HTML se utilizó para la estructura y organización del contenido del sistema, permitiendo la correcta visualización de la interfaz de usuario en distintos dispositivos. Gracias a su simplicidad y compatibilidad con otras tecnologías, se logró desarrollar una base sólida para la presentación de la información. (Delgado, 2024) (vadavo, 2024)



3.3 CASCADING STYLE SHEETS

fue desarrollado en 1996 por Håkon Wium Lie y Bert Bos como una forma de separar el contenido (HTML) de la presentación visual en las páginas web.

CSS es un lenguaje de estilo utilizado para describir la presentación de un documento HTML. Permite controlar el diseño, el color, las fuentes y otros aspectos visuales de una página web.

Características clave:

- **Diseño visual:** Permite definir el aspecto de los elementos HTML, como el tamaño, color, y disposición.
- **Estilos responsivos:** Con CSS, se puede crear un diseño que se adapte a diferentes tamaños de pantalla utilizando técnicas como los medios de consulta (media queries).
- **Animaciones y transiciones:** Permite agregar efectos visuales para mejorar la experiencia del usuario.

En este proyecto, CSS se empleó para el diseño y la estilización de la interfaz, asegurando una experiencia visual atractiva y una disposición adecuada de los elementos en pantalla. Mediante el uso de estilos personalizados y técnicas de diseño responsivo, se garantizó una correcta adaptación del sistema a diferentes dispositivos y tamaños de pantalla. (Code, 2022) (Zermeño, 2022)



3.4 PHP: Lenguaje de Programación para Desarrollo Web

Es un lenguaje de programación de código abierto diseñado para el desarrollo web. Fue creado en 1994 por Rasmus Lerdorf y, desde entonces, se ha convertido en una de las tecnologías más utilizadas para la creación de sitios dinámicos y aplicaciones web.

Este lenguaje se ejecuta en el servidor, lo que permite generar contenido dinámico antes de enviarlo al navegador del usuario. Se integra fácilmente con bases de datos como MySQL, facilitando la creación de aplicaciones interactivas y sistemas de gestión de contenido (CMS).

Características Clave:

- **Código abierto y multiplataforma:** Compatible con sistemas como Windows, Linux y macOS.
- **Ejecución en el servidor:** Procesa el código antes de enviarlo al navegador, permitiendo la generación de contenido dinámico.
- **Interacción con bases de datos:** Soporte para MySQL, PostgreSQL, SQLite, entre otras.
- **Compatibilidad con HTML y JavaScript:** Se puede combinar con tecnologías frontend para mejorar la experiencia del usuario.

En este proyecto, PHP se utilizó para la creación del backend, permitiendo la conexión con la base de datos y la gestión de la lógica del sitio web. Gracias a su flexibilidad y eficiencia, se logró desarrollar un sistema dinámico y funcional. (Maldonado, 2021) (Argenis, 2024)



3.5 MySQL: Sistema de Gestión de Bases de Datos Relacionales

MySQL es un sistema de gestión de bases de datos relacionales (RDBMS) de código abierto que permite el almacenamiento, manipulación y recuperación eficiente de datos estructurados. Fue desarrollado en 1995 por Michael Widenius, David Axmark y Allan Larsson y es ampliamente utilizado en aplicaciones web, sistemas empresariales y plataformas en la nube.

Este sistema es compatible con SQL (Structured Query Language), lo que facilita la creación, modificación y consulta de bases de datos. Gracias a su escalabilidad y rendimiento, MySQL es una opción preferida para sitios web dinámicos y aplicaciones de gran demanda.

Características Clave:

- **Alta velocidad y eficiencia:** Optimizado para manejar grandes volúmenes de datos con un rendimiento superior.
- **Soporte para múltiples motores de almacenamiento:** InnoDB (con soporte para transacciones y claves foráneas), MyISAM (más rápido en lecturas), entre otros.
- **Seguridad y control de acceso:** Permite la gestión de permisos mediante usuarios y autenticación segura.
- **Integración con múltiples lenguajes:** Compatible con PHP, Python, Java, entre otros, facilitando el desarrollo de aplicaciones web.

En este proyecto, MySQL se utilizó para el almacenamiento y gestión de los datos del sistema, permitiendo consultas optimizadas y una integración eficiente con el backend desarrollado en PHP. Su estabilidad y facilidad de uso fueron clave para garantizar el correcto funcionamiento de la base de datos. (Robledano, openwebinars, 2019) (Luna, 2024)



3.6 XAMPP: Entorno de Desarrollo para Servidores Web

Es un paquete de software de código abierto que proporciona un entorno de desarrollo completo para la creación y prueba de aplicaciones web. Fue desarrollado por Apache Friends y está disponible para Windows, Linux y macOS.

El nombre XAMPP proviene de la combinación de los principales componentes que incluye:

- **X:** Multiplataforma (compatible con varios sistemas operativos).
- **A:** Apache, el servidor web más utilizado.
- **M:** MySQL o MariaDB, sistemas de gestión de bases de datos.
- **P:** PHP, lenguaje de programación para desarrollo web.
- **P:** Perl, otro lenguaje de scripting compatible.

Características Clave:

- **Fácil instalación y configuración:** Permite desplegar rápidamente un entorno de desarrollo.
- **Incluye Apache, MySQL/MariaDB, PHP y Perl:** Todo lo necesario para ejecutar aplicaciones web.
- **Panel de control intuitivo:** Facilita la gestión de servicios como el servidor web y la base de datos.
- **Compatible con phpMyAdmin:** Permite la administración de bases de datos a través de una interfaz gráfica.
- **Útil para pruebas locales:** Permite desarrollar y probar proyectos antes de su implementación en un servidor en línea.

En este proyecto, XAMPP se utilizó para crear un entorno de desarrollo local, permitiendo la ejecución del servidor web, la base de datos y los scripts en PHP. Gracias a su facilidad de uso, se pudo realizar pruebas y configuraciones sin necesidad de un servidor en producción. (Peña, 2022) (Cyberstream, 2024)



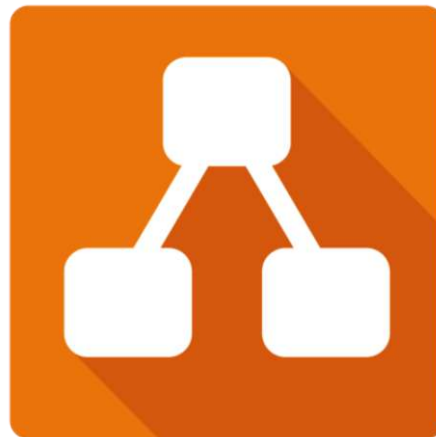
MODELADO DE LOS ARTEFACTOS DEL SISTEMA

Este capítulo aborda el desarrollo del modelo estructural y de comportamiento UML, permitiendo una representación clara y detallada de la arquitectura y funcionalidad del sistema. Se incluyen diagramas que describen tanto la estructura del sistema como la interacción entre los diferentes elementos.

Gran parte del desarrollo de este capítulo se realizó utilizando Draw.io, una herramienta en línea que permite la creación de diagramas de manera intuitiva y colaborativa. Se utilizó para diseñar el modelo estructural UML, que incluye el diagrama de clases, diagrama de objetos y diagrama de paquetes, facilitando la visualización de la estructura y organización del sistema. Además, se elaboró el modelado de comportamiento, representado por el diagrama de casos de uso y el diagrama de secuencia, los cuales describen la interacción de los usuarios con el sistema y el flujo de ejecución de los procesos principales.

Draw.io fue elegida por su interfaz intuitiva, la posibilidad de trabajo colaborativo en tiempo real y su capacidad para generar diagramas detallados con facilidad. Esto permitió organizar de manera clara los elementos del sistema, asegurando un diseño preciso y bien documentado.

A continuación, se adjunta el logo de la herramienta Draw.io, la cual fue utilizada en el desarrollo de este capítulo para crear los diagramas y modelos relacionados con la estructura y funcionamiento del sistema.



Para finalizar, se deja el enlace de la herramienta Draw.io para que pueda darle un vistazo y utilizarla según sus necesidades: <https://app.diagrams.net/>

4.1 LISTA DE REQUERIMIENTOS

NRO.	NOMNRE DEL REQUERIMIENTO	COMO PROBARLO (ATRIBUTOS DE CALIDAD)	PRIORIDAD
RF01.-	Gestión de Usuarios	- Registro de usuarios: Verificar que se puedan crear usuarios con ID, nombre de usuario, contraseña encriptada, intentos fallidos y último inicio de sesión. - Autenticación: Validar que los usuarios puedan iniciar sesión con su nombre de usuario y contraseña.	ALTA
RF02.-	Gestión de Personas	- Registro de personas: Verificar que se puedan registrar personas con ID, nombre, apellidos, dirección, teléfono, email, fecha de nacimiento y CI. - Asociación a oficinas: Comprobar que se pueda asignar una persona a una oficina específica.	ALTA
RF03.-	Gestión de Oficinas	- Registro de oficinas: Verificar que se puedan registrar oficinas con ID, nombre, dirección, teléfono y si es oficina central. - Búsqueda y filtrado: Comprobar que las oficinas puedan buscarse por nombre o dirección.	ALTA
RF04.-	Gestión de Cuentas	- Registro de cuentas: Verificar que se puedan registrar cuentas con ID, tipo de cuenta (ahorro/corriente), tipo de moneda (bolivianos/dólares), fecha de apertura, estado, número de cuenta y saldo inicial. - Asociación a clientes: Comprobar que cada cuenta esté vinculada a una persona registrada.	ALTA
RF05.-	Gestión de Tarjetas	- Emisión de tarjetas: Verificar que se puedan registrar tarjetas con ID, número de tarjeta, CVV, fecha de expiración, PIN y estado. - Asociación a cuentas: Comprobar que cada tarjeta esté vinculada a una cuenta registrada.	ALTA

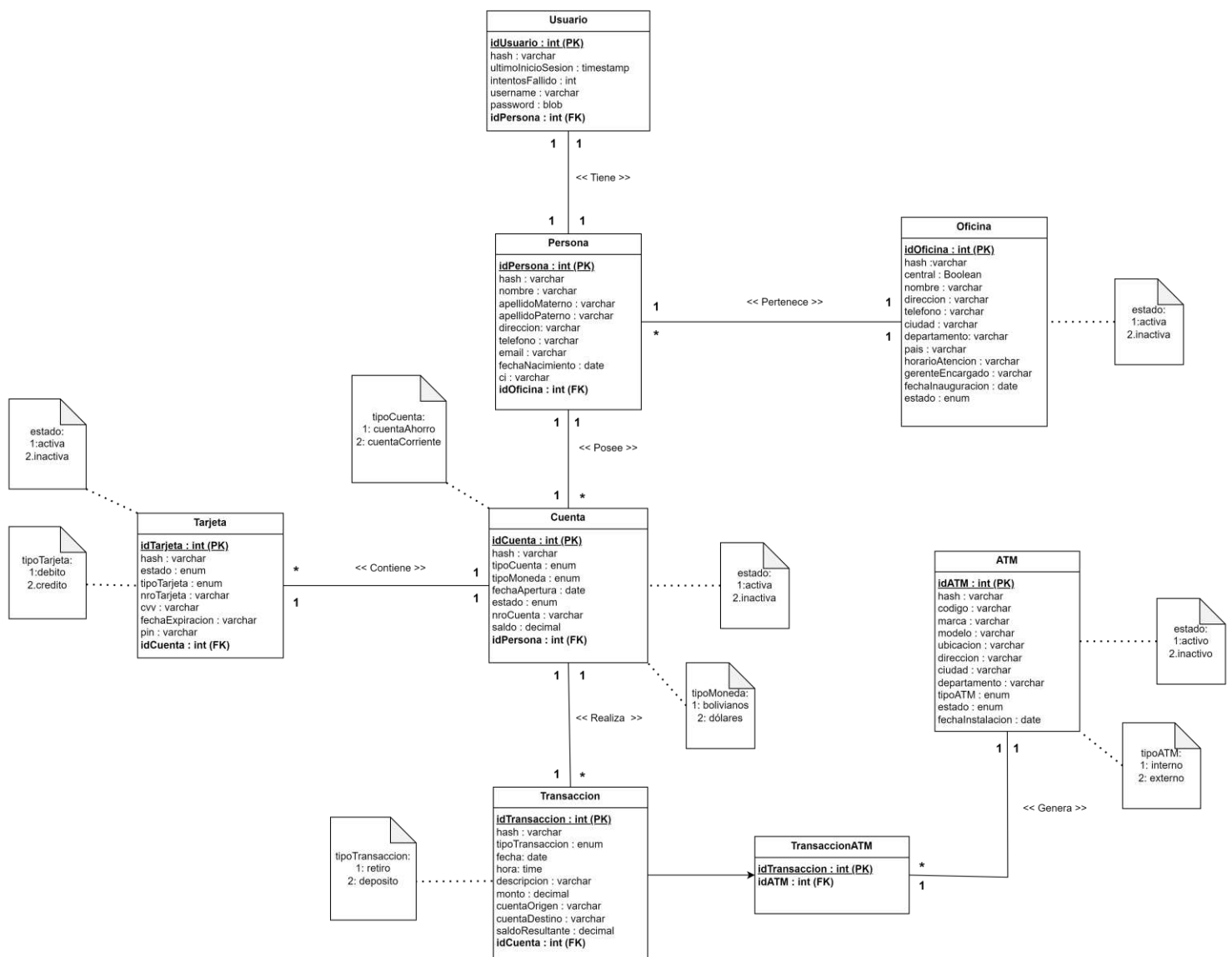
RF06.-	Gestión de Transacciones	- Registro de transacciones: Verificar que se puedan registrar transacciones con ID, tipo de transacción (retiro/depósito), fecha, hora, descripción y monto. - Asociación a cuentas: Comprobar que cada transacción esté vinculada a una cuenta registrada.	ALTA
RF07.-	Gestión de Cajeros Automáticos (ATM)	- Registro de cajeros: Verificar que se puedan registrar ATMs con ID y ubicación. - Asociación con transacciones: Comprobar que se pueda registrar qué transacciones fueron realizadas en cada ATM.	ALTA
RF08.-	Gestión de Transacciones en ATM	- Registro de transacciones ATM: Verificar que se puedan registrar transacciones realizadas en cajeros automáticos. - Asociación con cuentas y cajeros: Comprobar que cada transacción esté vinculada a una cuenta y a un ATM.	ALTA

4.2 MODELADO ESTRUCTURAL

El modelado estructural en UML se enfoca en representar la organización y la estructura estática de un sistema. Para ello, se utilizan diagramas como el Diagrama de Clases, que define las entidades, atributos y relaciones entre ellas; el Diagrama de Objetos, que muestra instancias concretas de las clases en un momento específico; y el Diagrama de Paquetes, que organiza y agrupa los elementos del sistema en módulos para mejorar su gestión y comprensión. Este enfoque permite visualizar la arquitectura del sistema, facilitando la comprensión y el diseño de la base de datos y otros componentes clave.

4.2.1 Diagrama de clases

Diagrama de Clases



4.2.2 Diagrama de objetos

Diagrama de Objetos

Cuenta 1 : * Tarjeta

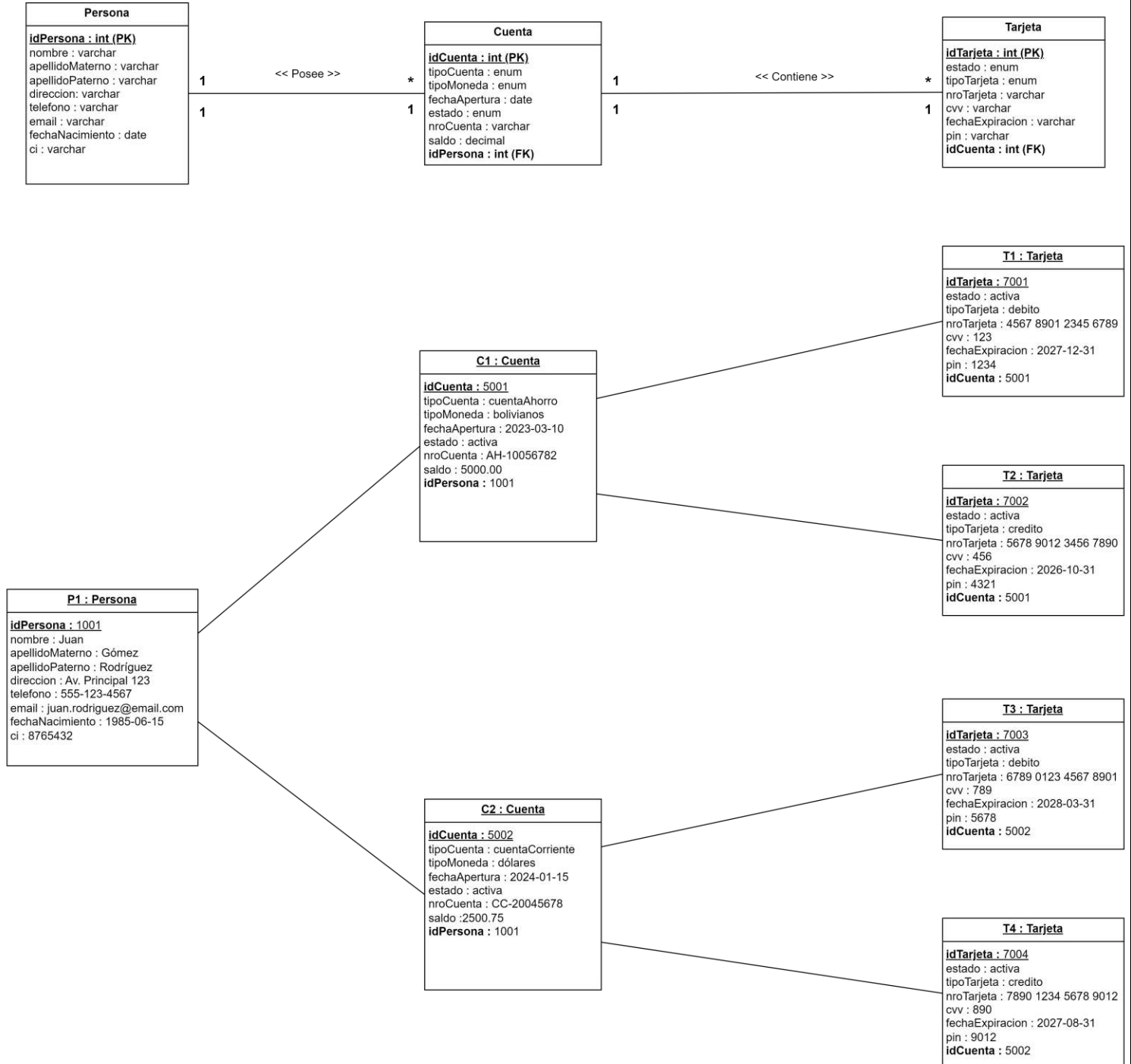
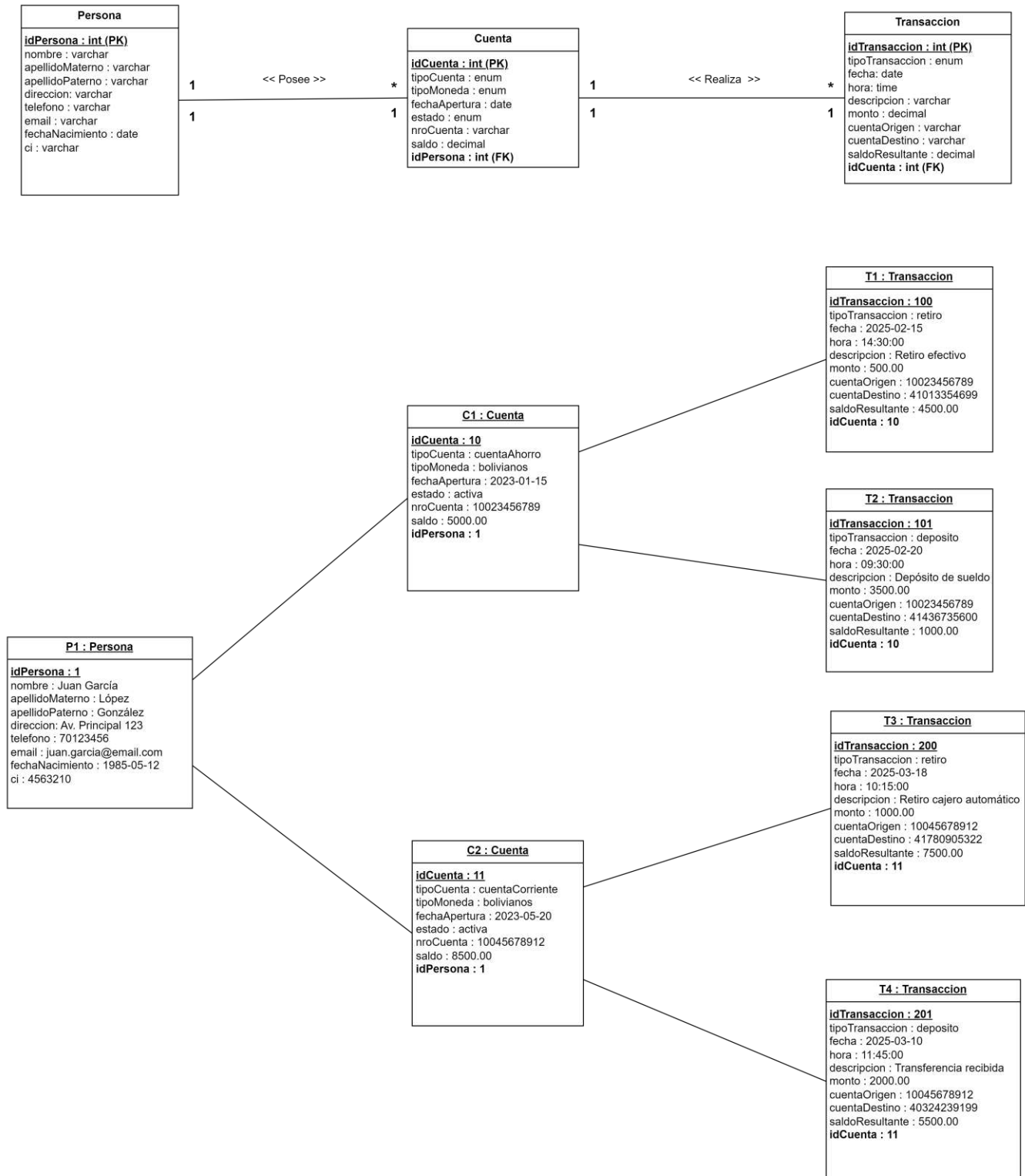


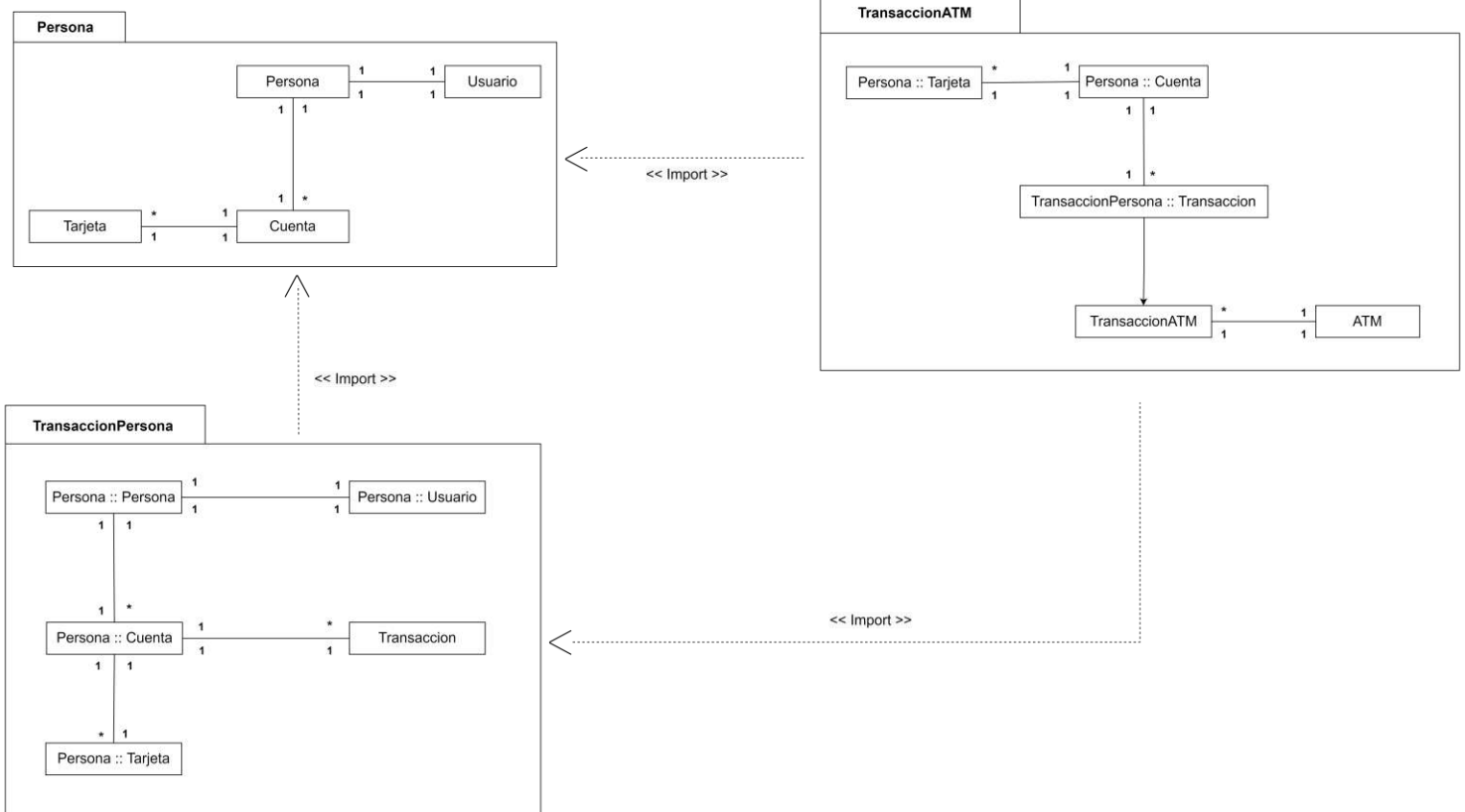
Diagrama de Objetos

Cuenta 1 : * Transaccion



4.2.3 Diagrama de paquetes

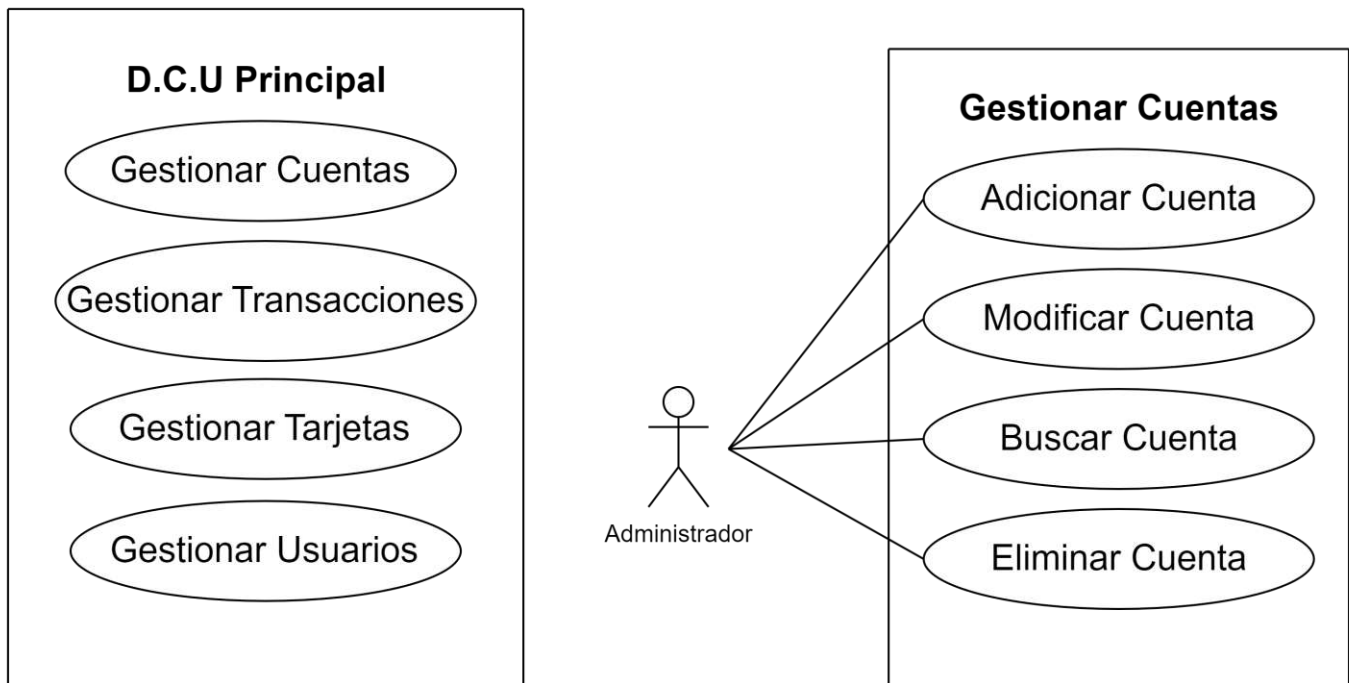
Diagrama de Paquetes

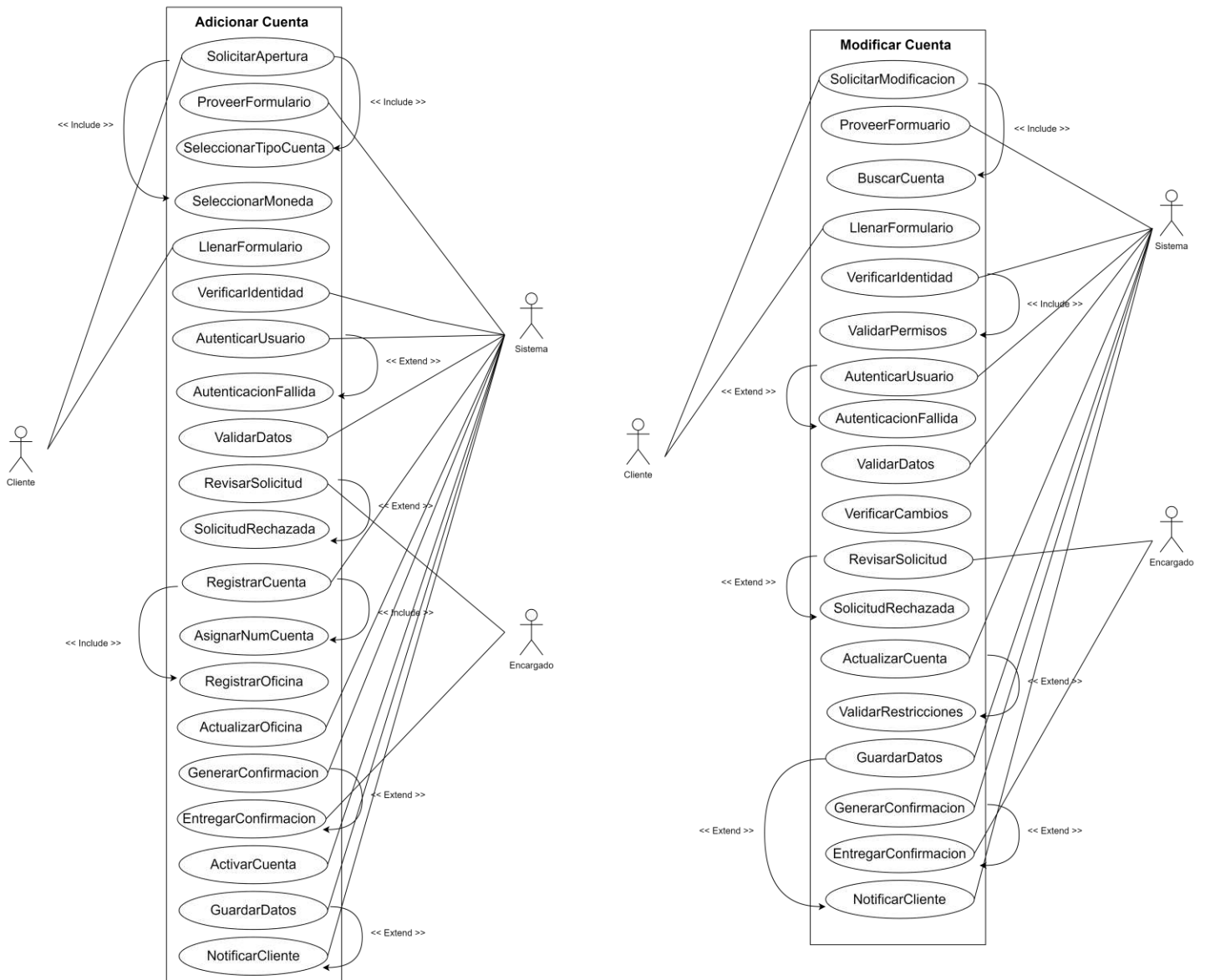


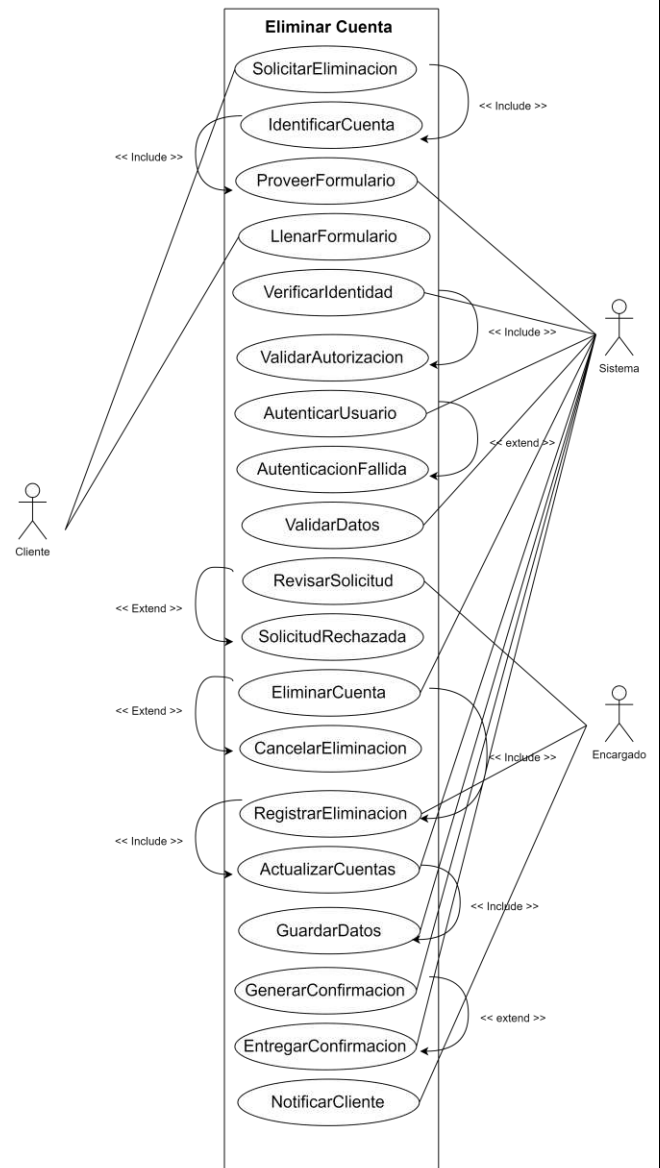
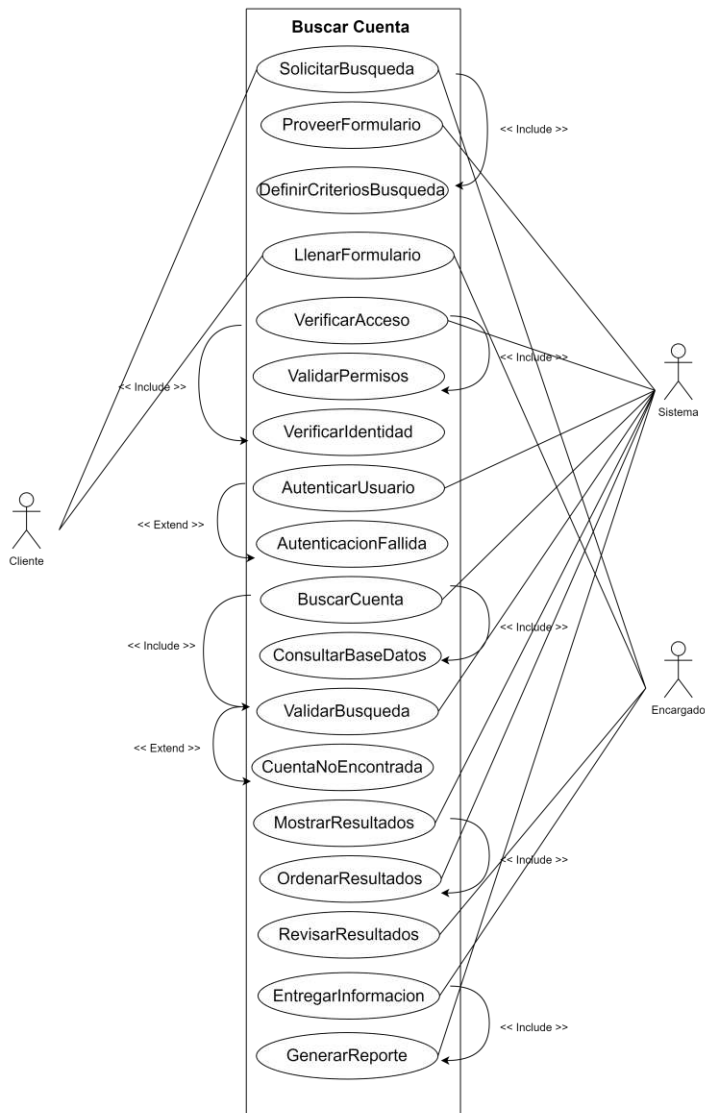
4.3 MODELADO DE COMPORTAMIENTO

El modelado de comportamiento en UML representa la dinámica del sistema, mostrando cómo interactúan sus elementos ante eventos y acciones. Para ello, se utilizan diagramas como el Diagrama de Casos de Uso, que define la relación entre actores y funcionalidades; el Diagrama de Secuencia, que describe el intercambio de mensajes entre objetos.

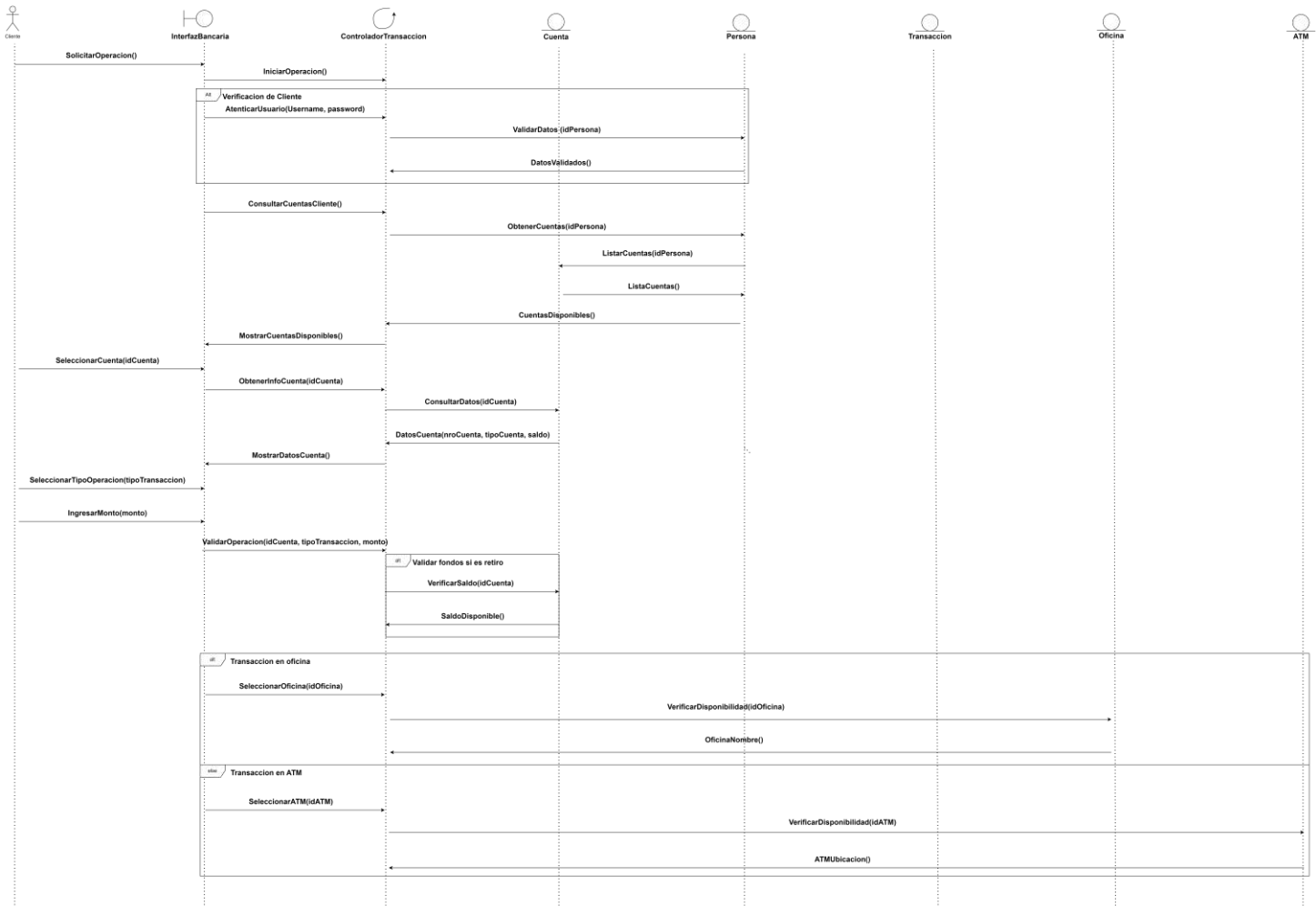
4.3.1 Diagrama de casos de uso

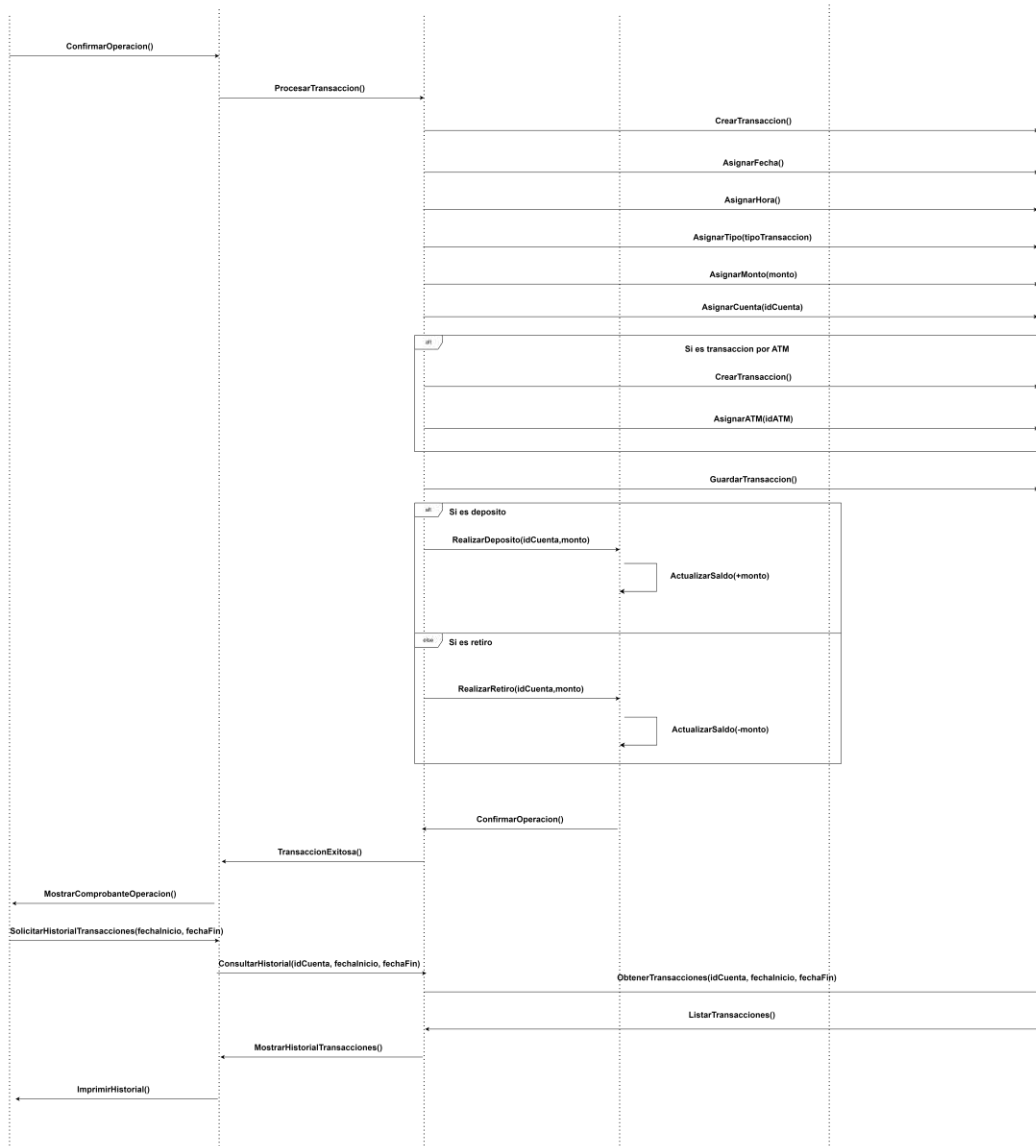






4.3.2 Diagrama de secuencia







DESARROLLO DE LA SISTEMA FUNCIONAL

1. Vista Bienvenida.php



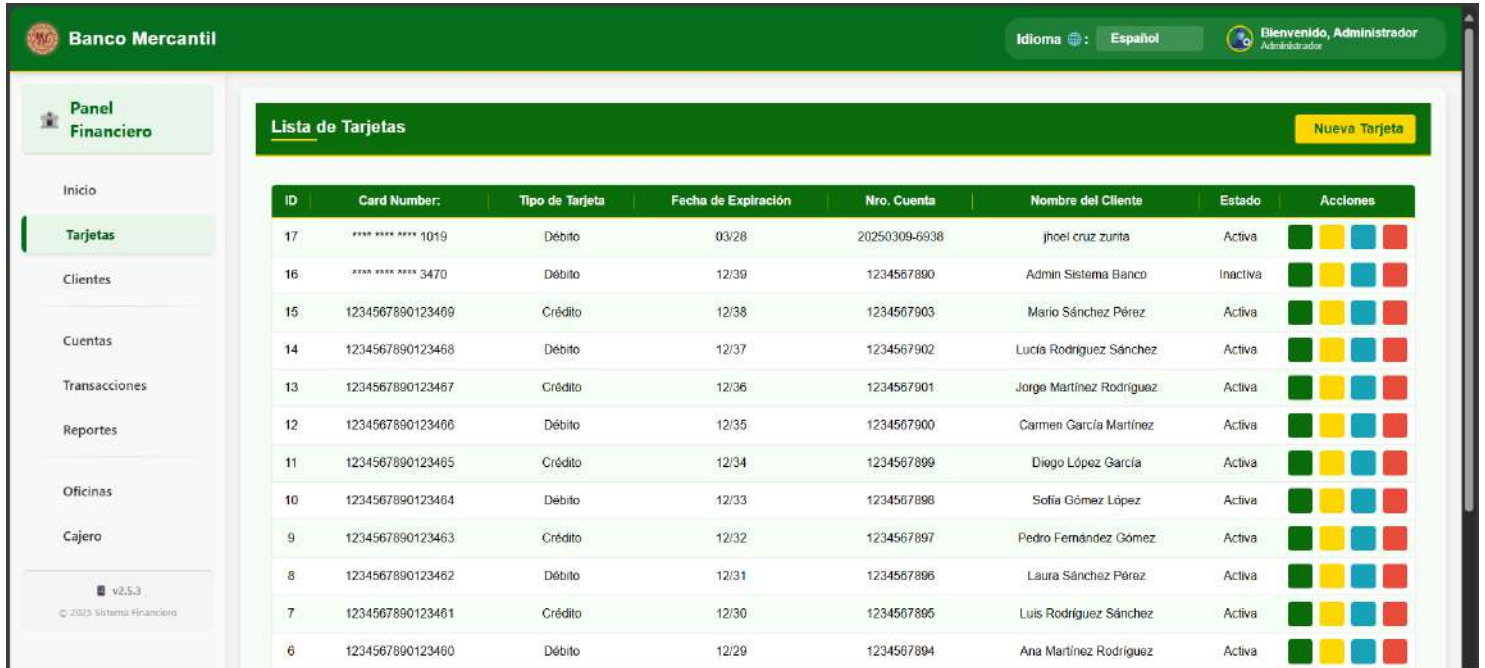
En la vista "Bienvenida", el sistema bancario presenta al usuario una interfaz elegante y profesional que muestra los valores fundamentales de Banco Mercantil. Esta pantalla inicial establece el tono para toda la experiencia del usuario, destacando los cinco pilares que sustentan la filosofía de la institución: Liderazgo, Servicio al cliente, Confiabilidad, Integridad y Profesionalismo.

La vista incorpora elementos visuales atractivos para cada valor, acompañados de descripciones concisas que transmiten la esencia de cada principio. Bajo el título "Nuestros Valores", se presenta el lema "Los pilares que guían nuestra misión de servicio", reforzando el compromiso del banco con la excelencia.

En la sección inferior, titulada "Nuestro Compromiso Contigo", se destacan cuatro servicios esenciales: Soporte 24/7, Máxima seguridad, Asesoría experta y Banca móvil, con la promesa de acompañamiento "en cada paso de tu camino financiero". Tres botones amarillos enfatizan los beneficios principales: Atención personalizada, Seguridad garantizada y Crecimiento sostenible.

La estructura MVC (Modelo-Vista-Controlador) permite una navegación fluida mediante un menú lateral que facilita el acceso a las diferentes funcionalidades del sistema, mientras el diseño en tonos verdes transmite estabilidad y confianza, valores esenciales para una entidad financiera.

2. Vista Listar.php



ID	Card Number	Tipo de Tarjeta	Fecha de Expiración	Nro. Cuenta	Nombre del Cliente	Estado	Acciones
17	**** * 1019	Débito	03/28	20250309-6938	jhoel cruz zunta	Activa	
16	**** * 3470	Débito	12/39	1234567890	Admin Sistema Banco	Inactiva	
15	1234567890123469	Crédito	12/38	1234567903	Mario Sánchez Pérez	Activa	
14	1234567890123468	Débito	12/37	1234567902	Lucia Rodriguez Sánchez	Activa	
13	1234567890123467	Crédito	12/36	1234567901	Jorge Martínez Rodríguez	Activa	
12	1234567890123466	Débito	12/35	1234567900	Carmen García Martínez	Activa	
11	1234567890123465	Crédito	12/34	1234567899	Diego López García	Activa	
10	1234567890123464	Débito	12/33	1234567898	Sofía Gómez López	Activa	
9	1234567890123463	Crédito	12/32	1234567897	Pedro Fernández Gómez	Activa	
8	1234567890123462	Débito	12/31	1234567896	Laura Sánchez Pérez	Activa	
7	1234567890123461	Crédito	12/30	1234567895	Luis Rodríguez Sánchez	Activa	
6	1234567890123460	Débito	12/29	1234567894	Ana Martínez Rodríguez	Activa	

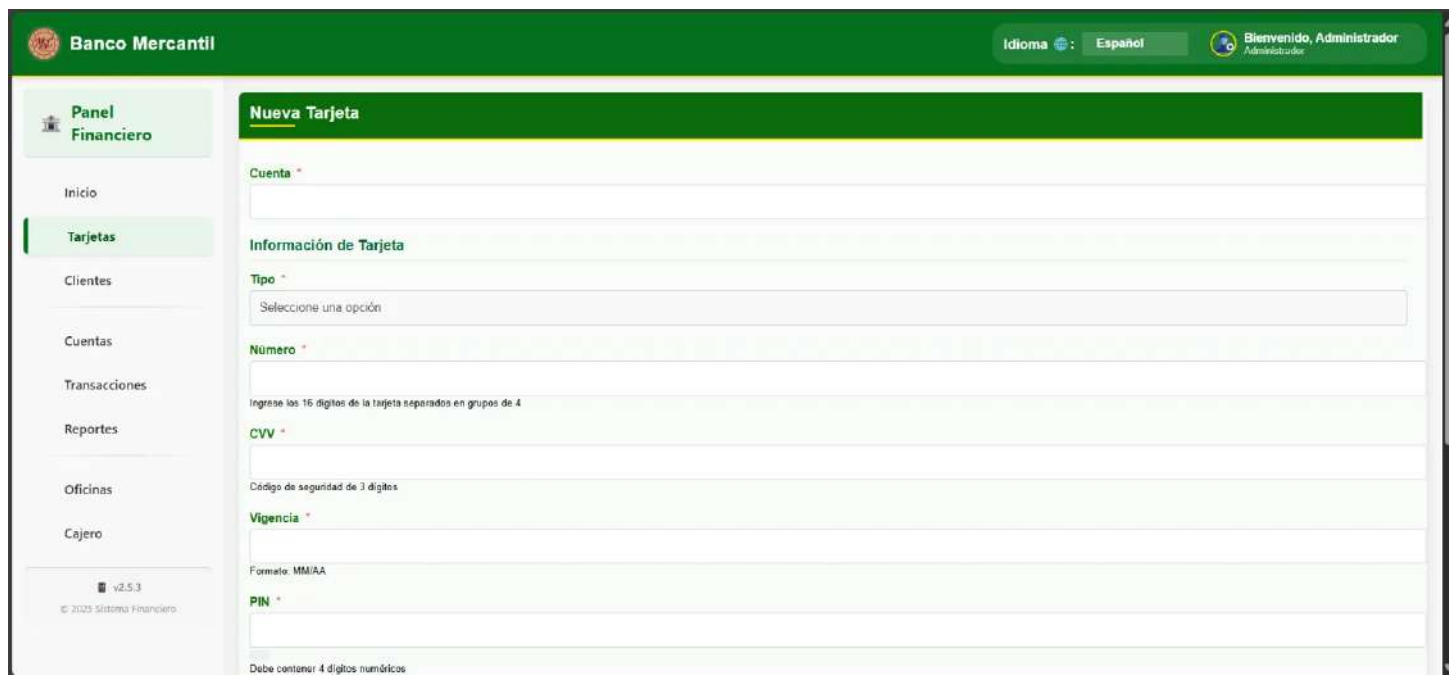
En la vista "Listar Tarjetas", el sistema bancario presenta una tabla completa con todas las tarjetas registradas, implementando medidas de seguridad esenciales para proteger la información sensible de los clientes. La interfaz muestra de manera organizada los registros de tarjetas bancarias.

La tabla está estructurada con columnas claramente definidas: ID, Card Number, Tipo de Tarjeta, Fecha de Expiración, Nro. Cuenta, Nombre del Cliente, Estado y Acciones. Las tarjetas se categorizan como Débito o Crédito y su estado (Activa/Inactiva) se indica con un código de colores intuitivo. Cada registro incluye un conjunto de botones de acción representados por iconos de colores que permiten ver detalles, editar información, cambiar el estado o eliminar registros según sea necesario.

En la parte superior de la tabla, destaca un botón amarillo "Nueva Tarjeta" que facilita la adición de nuevos registros al sistema. La interfaz mantiene la coherencia con el resto del Panel Financiero mediante el menú lateral que permite la navegación entre las diferentes secciones del sistema bancario (Inicio, Tarjetas, Clientes, Cuentas, Transacciones, Reportes, Oficinas y Cajero).

Este diseño de vista, alineado con la arquitectura MVC, proporciona a los administradores una herramienta eficiente para gestionar las tarjetas bancarias mientras mantiene los estándares de seguridad necesarios para operaciones financieras.

3. Vista CrearTarjeta.php



En la vista "Nueva Tarjeta", el sistema bancario presenta un formulario estructurado para la creación de nuevas tarjetas bancarias. Esta interfaz ha sido diseñada para guiar al administrador a través del proceso de emisión de tarjetas de manera intuitiva y segura, cumpliendo con los estándares del sector financiero.

La pantalla se organiza en campos obligatorios claramente identificados con asteriscos (*). Comienza solicitando la selección de una "Cuenta" asociada, campo fundamental para vincular la tarjeta con el cliente correspondiente. Bajo el encabezado "Información de Tarjeta", se requiere especificar el "Tipo" de tarjeta mediante un menú desplegable que permite elegir entre las opciones disponibles (débito o crédito).

Para el campo "Número", a diferencia de la generación automática, esta versión del formulario solicita al usuario ingresar manualmente los 16 dígitos del número de tarjeta, con la instrucción específica: "Ingrese los 16 dígitos de la tarjeta separados en grupos de 4". Esta modificación otorga mayor control al administrador en la asignación de números de tarjeta.

Los elementos de seguridad incluyen el campo "CVV", acompañado de la descripción "Código de seguridad de 3 dígitos", y el campo "Vigencia" que debe completarse en formato "MM/AA" según indica la aclaración. El formulario finaliza con la solicitud del "PIN", que "Debe contener 4 dígitos numéricos", garantizando así el cumplimiento de los protocolos de seguridad bancaria.

4. Vista Editar Tarjeta

**Banco Mercantil**

Idioma  : Español

Bienvenido, Administrador

Panel Financiero

Inicio

Tarjetas

Cuentas

Transacciones

Reportes

Oficinas

Cajero

v2.5.3
© 2025 Sistema Financiero

Editar Datos de la Tarjeta

Editando Tarjeta #16



Tarjeta de Débito

Activa

Hash de seguridad: a7f04de2c...

Verificar

Límites

Bloquear

Fecha de emisión: 05/03/2023

Sucursal: Casa Matriz

Uso Internacional: Habilitado

Seguridad: Estándar PCI DSS

Información de la Cuenta

Cuenta Asociada *

Juan Pérez - 1234567904 (\$)

Esta cuenta será asociada a la tarjeta para transacciones

Información de Tarjeta

Tipo de Tarjeta *

Débito

Fecha de Expiración *

12/39

Card Number: *

PIN *

**Banco Mercantil**

Idioma  : Español

Bienvenido, Administrador

Oficinas

Cajero

v2.5.3
© 2025 Sistema Financiero

Esta cuenta será asociada a la tarjeta para transacciones

Información de Tarjeta

Tipo de Tarjeta *

Débito

Fecha de Expiración *

12/39

Card Number: *

4532 7854 9632 3470

CVV *

123

PIN *

Estado *

✓ Activa

✗ Inactiva

Medidas de Seguridad

Última Modificación

09/03/2025 06:56:49

Aviso de Seguridad

Todo cambio en la información de la tarjeta quedará registrado y pedirá requerir verificación adicional. El cambio del PIN o bloqueo de la tarjeta se aplicará inmediatamente.

Guardar Cambios

Cancelar

Bloquear Tarjeta

View History

En la vista "Editar Datos de la Tarjeta", el sistema bancario ofrece una interfaz completa que permite a los administradores gestionar y modificar los detalles de tarjetas existentes. Esta pantalla combina elementos visuales e informativos para facilitar la administración segura de los instrumentos financieros.

La interfaz está encabezada por un indicador claro que muestra que se está editando la Tarjeta #16, acompañado de una representación gráfica de la tarjeta de débito que incluye el logo del Banco Mercantil, los últimos cuatro dígitos del número (3470) y la fecha de vencimiento (12/39). Por motivos de seguridad, los primeros doce dígitos aparecen enmascarados con asteriscos, permitiendo identificar la tarjeta sin comprometer información sensible.

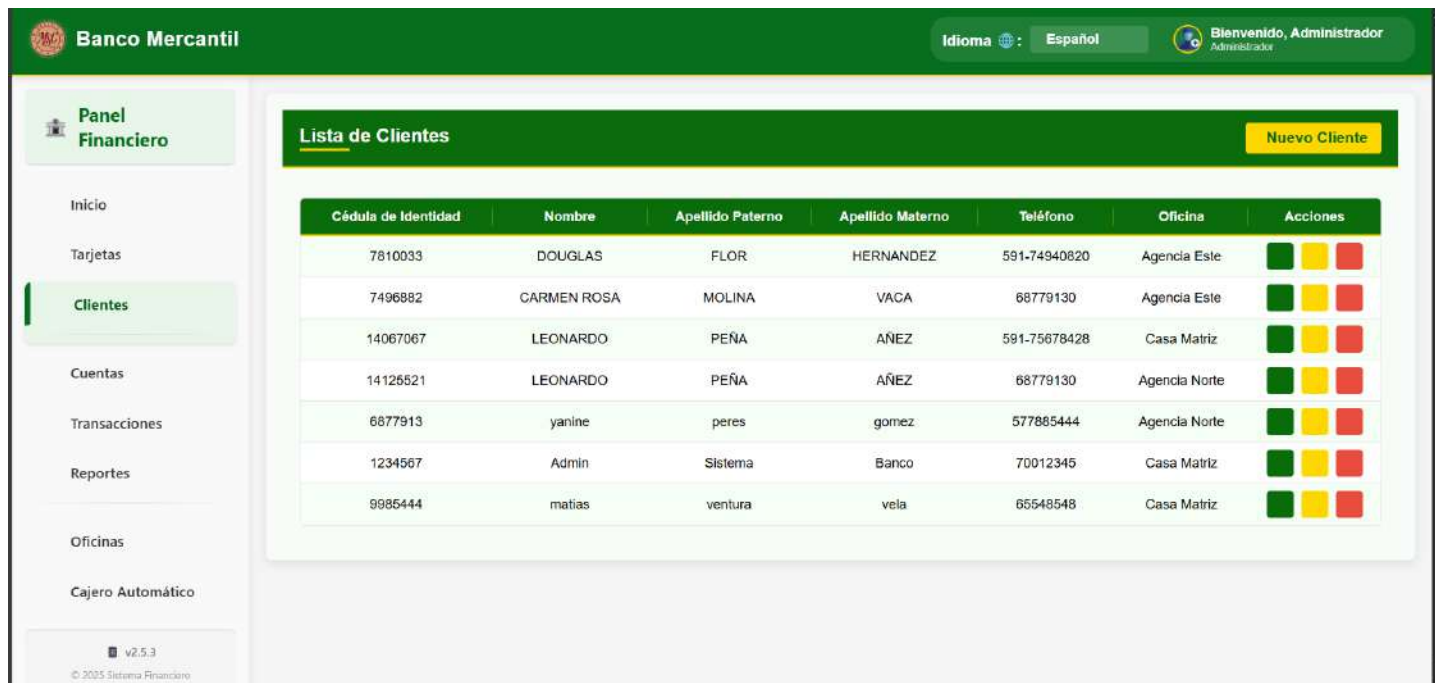
La vista se organiza en secciones bien estructuradas. En la parte superior se muestra la información general de la tarjeta: su tipo (Débito), estado actual (Activa indicada con un punto verde), hash de seguridad parcial, fecha de emisión (05/03/2023), sucursal emisora (Casa Matriz), configuración para uso internacional (Habilitado) y el estándar de seguridad implementado (Estándar PCI DSS). Tres botones funcionales permiten "Verificar", ajustar "Límites" o "Bloquear" la tarjeta según se requiera.

En la sección "Información de la Cuenta", se muestra la cuenta asociada perteneciente a Juan Pérez (1234567904 \$), con una nota aclaratoria indicando que esta cuenta será utilizada para las transacciones realizadas con la tarjeta.

La sección "Información de Tarjeta" permite modificar elementos clave como el tipo de tarjeta, la fecha de expiración en formato MM/YY y el número completo (5532 7964 9632 3470). Los campos sensibles como el PIN aparecen enmascarados y se incluye el CVV (123) con su descripción correspondiente. El estado de la tarjeta puede cambiarse mediante botones de selección entre Activa e Inactiva.

La parte inferior presenta las "Medidas de Seguridad", mostrando la fecha y hora de la última modificación (09/03/2023 08:26:49) y un aviso importante sobre la verificación de cambios. Los botones de acción ("Guardar Cambios", "Cancelar", "Bloquear Tarjeta" y "Ver Historial") facilitan la gestión completa del ciclo de vida de la tarjeta.

5. Vista Listar Clientes



Cédula de Identidad	Nombre	Apellido Paterno	Apellido Materno	Teléfono	Oficina	Acciones
7810033	DOUGLAS	FLOR	HERNANDEZ	591-74940820	Agencia Este	  
7496882	CARMEN ROSA	MOLINA	VACA	68779130	Agencia Este	  
14067067	LEONARDO	PEÑA	AÑEZ	591-75678428	Casa Matriz	  
14126521	LEONARDO	PEÑA	AÑEZ	68779130	Agencia Norte	  
6877913	yanine	peres	gomez	577865444	Agencia Norte	  
1234567	Admin	Sistema	Banco	70012345	Casa Matriz	  
9985444	matias	ventura	vela	65548548	Casa Matriz	  

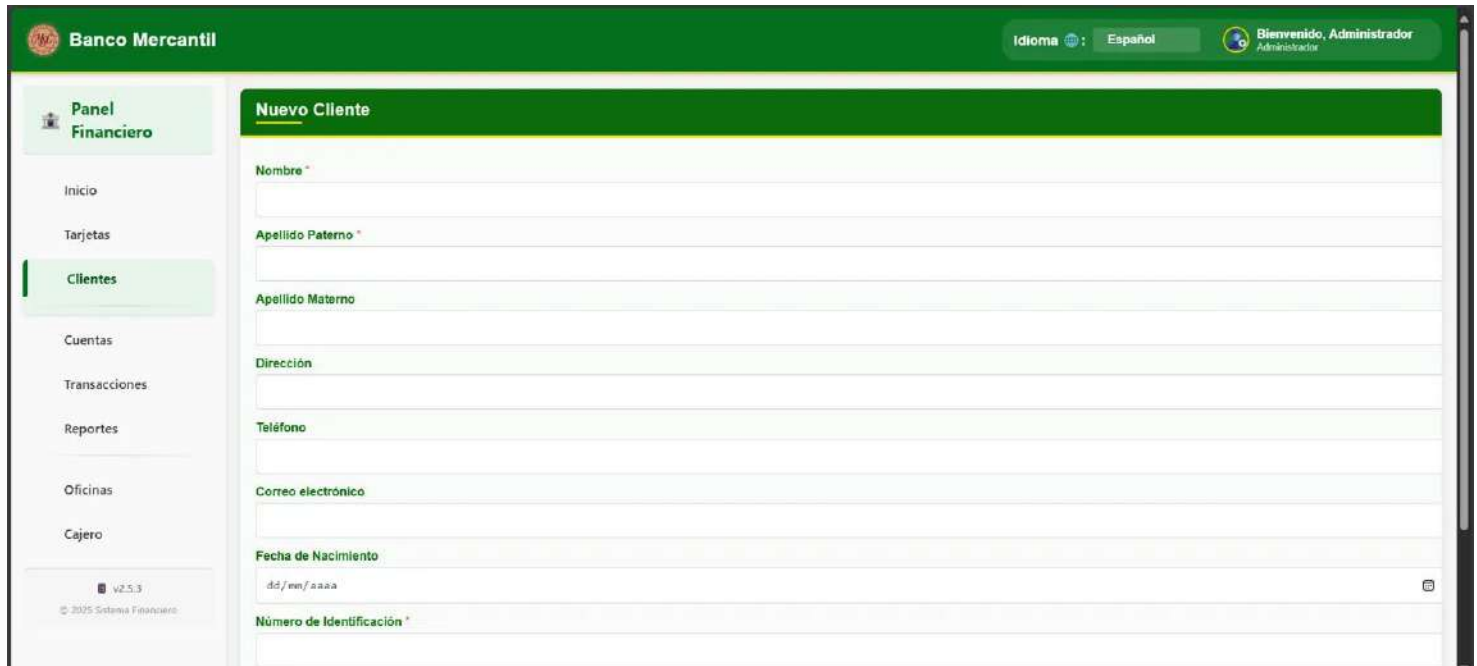
. La vista "Lista de Clientes" del Banco Mercantil presenta una interfaz organizada que muestra la base de clientes registrados en el sistema financiero. La tabla incluye información esencial como Cédula de Identidad, Nombre completo (separado en Nombre, Apellido Paterno y Apellido Materno), Teléfono y Oficina asociada al cliente.

El diseño mantiene la coherencia con el resto del sistema, destacando el botón "Nuevo Cliente" en amarillo que permite a los administradores registrar nuevos usuarios en la plataforma. Cada registro de cliente cuenta con tres botones de acción representados por íconos de colores que facilitan la gestión de la información.

Los datos se encuentran distribuidos entre diferentes sucursales (Casa Matriz, Agencia Este y Agencia Norte), proporcionando una visión completa de la cartera de clientes del banco. Esta vista permite a los administradores acceder rápidamente a la información de contacto y localización de cada cliente, facilitando la gestión de relaciones y servicios bancarios.

La interfaz se integra perfectamente con el menú lateral de navegación del Panel Financiero, permitiendo al usuario desplazarse eficientemente entre las distintas funcionalidades del sistema bancario.

6. Vista Crear Cliente

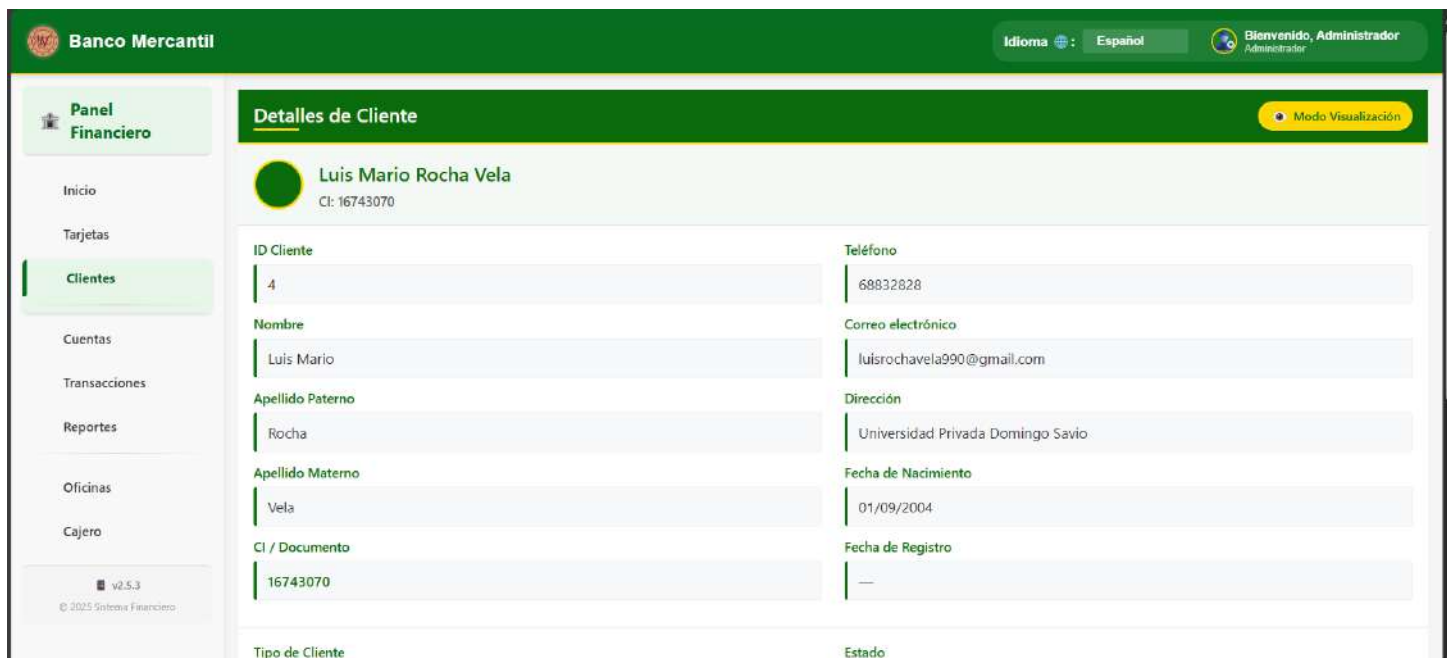


La vista "Nuevo Cliente" presenta un formulario estructurado para el registro de clientes en el sistema del Banco Mercantil. La interfaz solicita información personal con campos claramente identificados, donde los obligatorios están marcados con asterisco (*).

El formulario solicita datos esenciales como Nombre y Apellido Paterno (ambos obligatorios), Apellido Materno (opcional), Dirección completa, Teléfono de contacto y Correo electrónico para comunicaciones. Incluye también un campo para la Fecha de Nacimiento con formato específico (dd/mm/aaaa) y un campo obligatorio para el Número de Identificación, esencial para los procesos de verificación bancaria.

Este diseño minimalista facilita el proceso de registro de nuevos usuarios en el sistema bancario, manteniendo la coherencia visual con el resto de la plataforma y permitiendo una experiencia fluida para los administradores. La distribución vertical de los campos optimiza el espacio y facilita la captura progresiva de información, siguiendo un orden lógico que refleja las mejores prácticas en la gestión de datos de clientes en entidades financieras.

7. Vista Ver Detalles Cliente



The screenshot displays the 'Banco Mercantil' web application interface. The top navigation bar includes the bank's logo, the name 'Banco Mercantil', a language selector set to 'Español', and a user greeting 'Bienvenido, Administrador'. A left sidebar menu contains options like 'Inicio', 'Tarjetas', 'Clientes' (highlighted), 'Cuentas', 'Transacciones', 'Reportes', 'Oficinas', and 'Cajero'. The main content area is titled 'Detalles de Cliente' and features a profile card for 'Luis Mario Rocha Vela' with CI: 16743070. Below the card, a table lists client details in two columns. A 'Modo Visualización' button is located in the top right corner of the details section.

Detalles de Cliente	
ID Cliente	Teléfono
4	68832828
Nombre	Correo electrónico
Luis Mario	luisrochavela990@gmail.com
Apellido Paterno	Dirección
Rocha	Universidad Privada Domingo Savio
Apellido Materno	Fecha de Nacimiento
Vela	01/09/2004
CI / Documento	Fecha de Registro
16743070	—
Tipo de Cliente	Estado

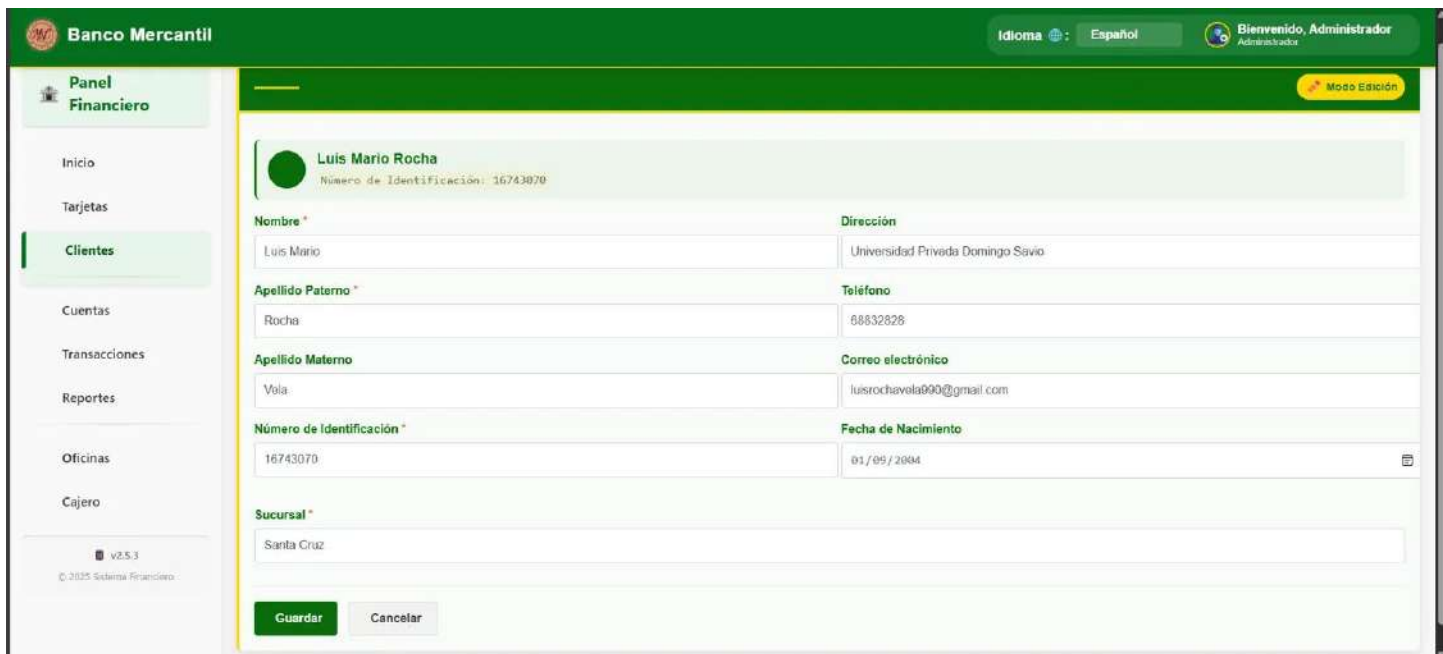
La vista "Detalles de Cliente" presenta una interfaz organizada que muestra la información completa de un cliente específico del Banco Mercantil. En este caso, se visualizan los datos de Luis Mario Rocha Vela, identificado con CI: 16743070.

La pantalla está encabezada por el nombre completo del cliente junto con un círculo verde que contiene sus iniciales, proporcionando un elemento visual que personaliza la experiencia. La información se estructura en dos columnas para optimizar el espacio y facilitar la lectura.

En la columna izquierda se muestran los identificadores del cliente (ID Cliente: 4), datos personales como Nombre (Luis Mario), Apellido Paterno (Rocha), Apellido Materno (Vela) y su número de Cédula de Identidad (16743070). La columna derecha complementa con información de contacto: Teléfono (68832828), Correo electrónico (luisrochavela990@gmail.com), Dirección (Universidad Privada Domingo Savio), Fecha de Nacimiento (01/09/2004) y Fecha de Registro.

La interfaz incluye un botón "Modo Visualización" en la esquina superior derecha que permite alternar entre diferentes formas de ver la información. Esta vista proporciona a los administradores del sistema una visión completa y ordenada de los datos del cliente, facilitando la gestión y atención personalizada dentro del Panel Financiero.

8. Vista editar datos Cliente



The screenshot shows the 'Banco Mercantil' interface. On the left is a sidebar with a 'Panel Financiero' menu containing links to Inicio, Tarjetas, Clientes (highlighted), Cuentas, Transacciones, Reportes, Oficinas, and Cajero. At the bottom of the sidebar, it says 'v2.5.3' and '© 2025 Sistema Financiero'. The main area has a green header with 'Banco Mercantil', a language selector set to 'Español', and a user greeting 'Bienvenido, Administrador'. A yellow 'Modo Edición' button is in the top right. The form is for 'Luis Mario Rocha' with ID '16743070'. It contains several input fields: 'Nombre *' (Luis Mario), 'Dirección' (Universidad Privada Domingo Savio), 'Apellido Paterno *' (Rocha), 'Teléfono' (68832828), 'Apellido Materno' (Vela), 'Correo electrónico' (luisrochavela990@gmail.com), 'Número de Identificación *' (16743070), 'Fecha de Nacimiento' (01/09/2004), and 'Sucursal *' (Santa Cruz). At the bottom are 'Guardar' and 'Cancelar' buttons.

La vista "Editar Cliente" del Banco Mercantil presenta un formulario interactivo que permite modificar la información del cliente Luis Mario Rocha (identificación: 16743070). Esta interfaz, activada mediante el "Modo Edición", transforma los campos informativos en elementos editables para actualizar los datos según sea necesario.

El formulario está organizado en dos columnas que equilibran visualmente la información. Los campos obligatorios están marcados con asterisco (*), indicando información crítica que debe mantenerse: Nombre, Apellido Paterno y Número de Identificación. El sistema muestra claramente los datos actuales del cliente, permitiendo modificar información de contacto como Dirección (Universidad Privada Domingo Savio), Teléfono (68832828) y Correo electrónico (luisrochavela990@gmail.com).

La interfaz incluye campos para datos personales adicionales como Apellido Materno (Vela) y Fecha de Nacimiento (01/09/2004), así como un campo obligatorio para seleccionar la Sucursal (Santa Cruz) asociada al cliente. En la parte inferior, se presentan dos botones de acción: "Guardar" (verde, destacando la acción principal) y "Cancelar" (gris, permitiendo abandonar los cambios).

Esta vista facilita la actualización eficiente de la información del cliente mientras mantiene la consistencia e integridad de los datos en el sistema bancario, siguiendo los estándares de seguridad y usabilidad necesarios para operaciones financieras.

9. Vista Listar Cuentas

Banco Mercantil

Idioma

Español

Bienvenido, Administrador

Administrador

Panel Financiero

Inicio

Tarjetas

Cientes

Cuentas

Transacciones

Reportes

Oficinas

Cajero

v2.5.3

© 2025 Sistema Financiero

Lista de Cuentas

Nueva Cuenta

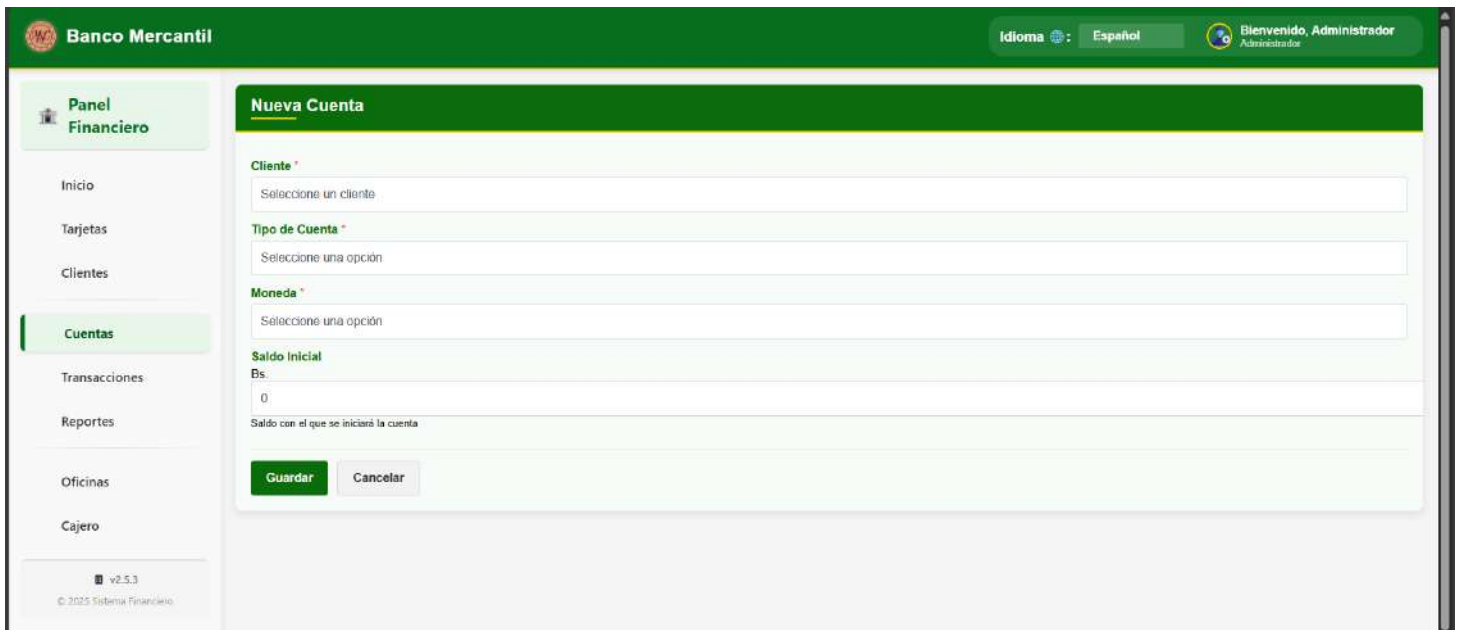
Nro. Cuenta	Cliente	Tipo de Cuenta	Moneda	Saldo	Fecha de Inauguración	Estado	Acciones
20250309-6938	Jhoel Cruz Zurita	Corriente	Dólares	\$ 11,120,061.00	09/03/2025	Inactiva	
20250309-4181	Carlos García Martínez	Corriente	Dólares	\$ 1,319,210.00	09/03/2025	Inactiva	
1234567904	Admin Sistema Banco	Corriente	Dólares	\$ 45,245.00	01/03/2021	Inactiva	
1234567903	Mario Sánchez Pérez	Corriente	Dólares	\$ 8,222.00	01/02/2021	Inactiva	
1234567902	Lucía Rodríguez Sánchez	Corriente	Dólares	\$ 6,500.00	01/01/2021	Inactiva	
1234567901	Jorge Martínez Rodríguez	Corriente	Dólares	\$ 6,000.00	01/12/2020	Inactiva	
1234567900	Carmen García Martínez	Corriente	Dólares	\$ 5,500.00	01/11/2020	Inactiva	
1234567899	Diego López García	Corriente	Dólares	\$ 5,000.00	01/10/2020	Inactiva	
1234567898	Sofía Gómez López	Corriente	Dólares	\$ 4,500.00	01/09/2020	Inactiva	
1234567897	Pedro Fernández Gómez	Corriente	Dólares	\$ 4,000.00	01/08/2020	Inactiva	
1234567896	Laura Sánchez Pérez	Corriente	Dólares	\$ 3,500.00	01/07/2020	Inactiva	
1234567895	Luis Rodríguez Sánchez	Corriente	Dólares	\$ 3,000.00	01/06/2020	Inactiva	

La vista "Lista de Cuentas" del Banco Mercantil presenta una tabla organizada con todas las cuentas bancarias registradas en el sistema. La interfaz muestra información crucial como número de cuenta, nombre del titular, tipo de cuenta (todas son Corrientes), moneda (Dólares), saldo actual, fecha de inauguración y estado (todas aparecen como Inactivas).

Destaca la presencia de cuentas con saldos significativos, como la de Jhoel Cruz Zurita con más de \$11 millones y Carlos García Martínez con más de \$1.3 millones. Las cuentas están ordenadas cronológicamente por fecha de apertura, con las más recientes al inicio. Cada registro incluye botones de acción para gestionar las cuentas individuales.

En la parte superior, un botón amarillo "Nueva Cuenta" permite crear registros adicionales, manteniendo la coherencia con otras interfaces del sistema financiero.

10. Vista Crear Cuenta



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

Nueva Cuenta

Cliente *
Seleccione un cliente

Tipo de Cuenta *
Seleccione una opción

Moneda *
Seleccione una opción

Saldo Inicial
Bs.
0
Saldo con el que se iniciará la cuenta

Guardar **Cancelar**

v2.5.3
© 2025 Sistema Financiero

La vista "Nueva Cuenta" del Banco Mercantil presenta un formulario conciso para la creación de cuentas bancarias. Con campos obligatorios marcados con asterisco (*), solicita seleccionar el Cliente titular, el Tipo de Cuenta y la Moneda mediante menús desplegables. Incluye un campo para definir el Saldo Inicial (en Bolívaes), con una nota aclaratoria sobre el valor con que se activará la cuenta. La interfaz finaliza con botones de "Guardar" y "Cancelar", manteniendo el diseño limpio y funcional característico del sistema.

11. Vista ver detalles Cuenta



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

Detalles de Cuenta Modo Visualización

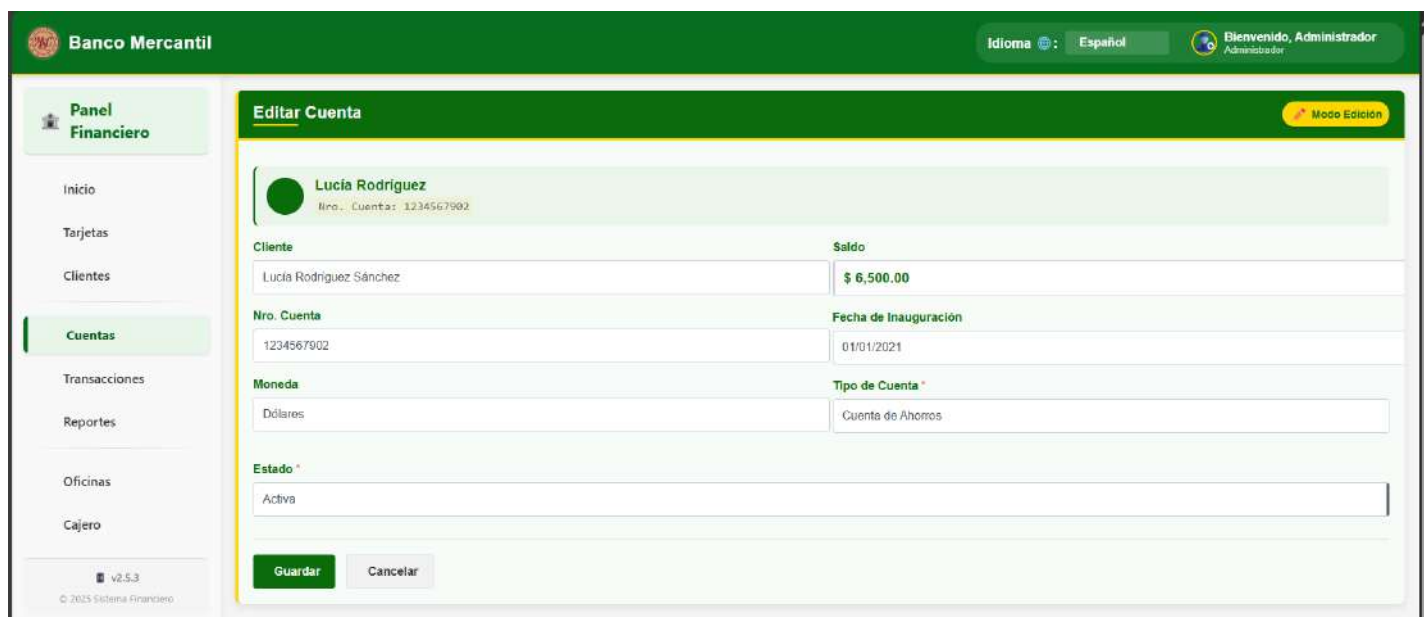
20250309-4181
\$ 1,319,210.00

Información de la Cuenta	Información del Cliente
Nro. Cuenta 20250309-4181	Nombre Carlos
Tipo de Cuenta Cuenta Corriente	Apellidos García Martínez
Moneda Dólares	CI / Documento 1234570
Saldo Actual \$ 1,319,210.00	Teléfono 70012348
Fecha de Inauguración 09/03/2025	Correo electrónico carlos.garcia@bancocamba.com

v2.5.3
© 2025 Sistema Financiero

La vista "Detalles de Cuenta" muestra información completa de la cuenta 20250309-4181 con saldo de \$1,319,210.00. La interfaz se divide en dos secciones: "Información de la Cuenta" e "Información del Cliente". En la primera sección se visualiza el número de cuenta, tipo (Cuenta Corriente), moneda (Dólares), saldo actual y fecha de inauguración (09/03/2025). La segunda sección presenta datos del titular: Carlos García Martínez, CI 1234570, teléfono 70012348 y correo electrónico. Un botón "Modo Visualización" en la esquina superior permite alternar la vista de la información.

12. Vista editar datos Cuenta



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

- Inicio
- Tarjetas
- Cientes
- Cuentas**
- Transacciones
- Reportes
- Oficinas
- Cajero

Editar Cuenta Modo Edición

Lucía Rodríguez
Nro. Cuenta: 1234567902

Cliente	Saldo
Lucía Rodríguez Sánchez	\$ 6,500.00
Nro. Cuenta	Fecha de Inauguración
1234567902	01/01/2021
Moneda	Tipo de Cuenta *
Dólares	Cuenta de Ahorros
Estado *	
Activa	

Guardar **Cancelar**

v2.5.3
© 2025 Sistema Financiero

La vista "Editar Cuenta" muestra un formulario para modificar la cuenta de Lucía Rodríguez (número 1234567902) con saldo de \$6,500.00. La interfaz permite actualizar campos como el Cliente (ya asignado a Lucía Rodríguez Sánchez), Número de Cuenta (no editable), Fecha de Inauguración (01/01/2021), Moneda (Dólares) y Tipo de Cuenta (Cuenta de Ahorros, marcado como campo obligatorio). El estado de la cuenta puede cambiarse a "Activa" mediante un campo obligatorio. La pantalla incluye botones de "Guardar" y "Cancelar" para confirmar o descartar los cambios, todo bajo el "Modo Edición" indicado en la esquina superior derecha.

13. Vista listar Transacciones

Banco Mercantil

Idioma

Español

Bienvenido, Administrador

Administrador

Panel Financiero

Inicio

Tarjetas

Cientes

Cuentas

Transacciones

Reportes

Oficinas

Cajero

v2.5.3

© 2025 Sistema Financiero

Lista de Transacciones

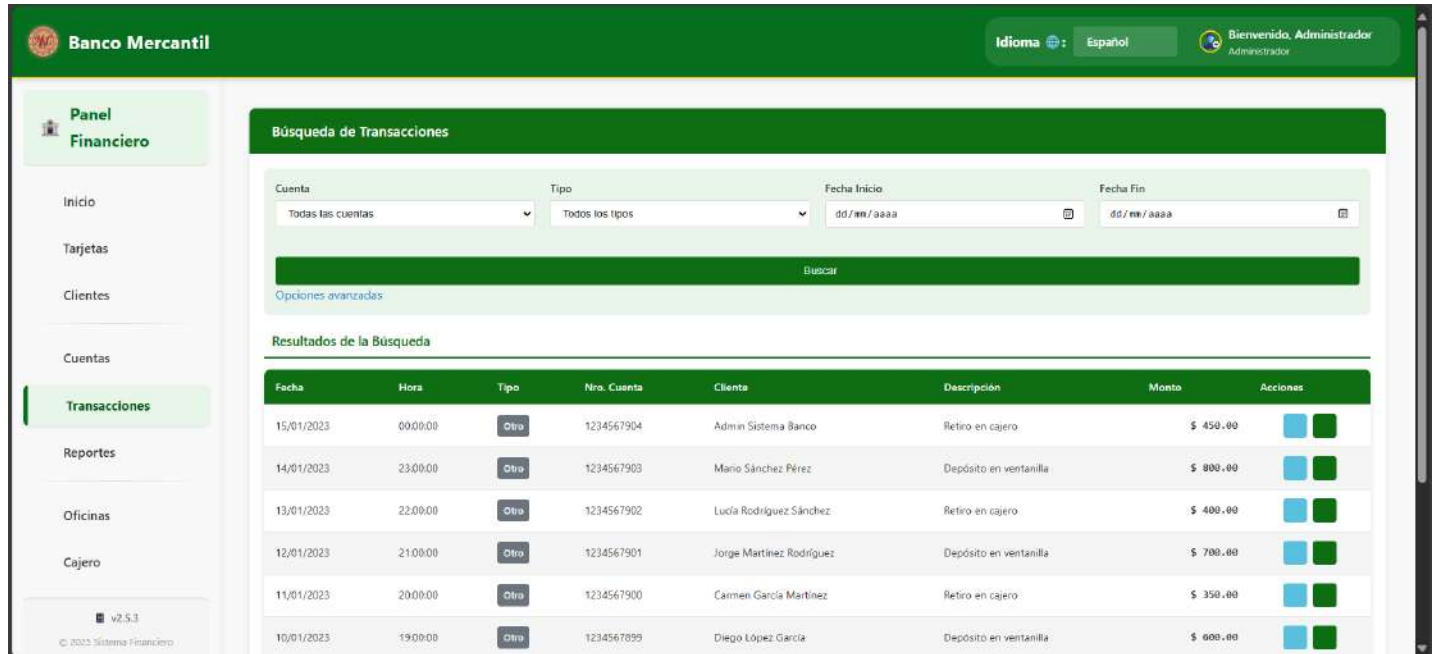
Buscar TransaccionesRetiroDepósitoImprimir

Fecha	Hora	Tipo	Nro. Cuenta	Cliente	Descripción	Monto	Acciones
10/03/2025	21:58:58	OTRO	1234567903	Mano Sánchez Pérez	Depósito de efectivo	\$ 1,222.00	
10/03/2025	21:21:08	OTRO	1234567904	Admin Sistema Banco	Retiro de efectivo	\$ 11.00	
10/03/2025	14:36:24	OTRO	20250309-4181	Carlos García Martínez	Retiro de efectivo	\$ 213.00	
10/03/2025	14:32:32	OTRO	1234567904	Admin Sistema Banco	Depósito	\$ 32,434.00	
10/03/2025	03:01:08	OTRO	1234567904	Admin Sistema Banco	Depósito	\$ 3,322.00	
10/03/2025	03:00:25	OTRO	20250309-4181	Carlos García Martínez	Depósito	\$ 323,423.00	
10/03/2025	02:09:21	OTRO	20250309-6938	jhoel cruz zurita	Depósito	\$ 11,111,111.00	
09/03/2025	21:26:53	OTRO	20250309-6938	jhoel cruz zurita	Retiro para la soda	\$ 50.00	
09/03/2025	21:24:39	OTRO	20250309-6938	jhoel cruz zurita	Retiro ATM	\$ 1,000.00	
09/03/2025	18:47:08	OTRO	20250309-6938	jhoel cruz zurita	Depósito Inicial	\$ 10,000.00	
09/03/2025	18:08:09	OTRO	20250309-4181	Carlos García Martínez	Retiro para mis brothers	\$ 2,000.00	

La vista "Lista de Transacciones" del Banco Mercantil muestra un registro cronológico de las operaciones financieras realizadas en el sistema. La tabla presenta información detallada incluyendo Fecha, Hora, Tipo (todas aparecen como "OTRO"), Número de Cuenta, Cliente, Descripción y Monto de cada transacción.

Destacan operaciones significativas como el depósito de \$11,111,111.00 de Jhoel Cruz Zurita y el depósito de \$323,423.00 de Carlos García Martínez, junto con transacciones menores como "Retiro para la soda" por \$50.00. La interfaz incluye botones funcionales en la parte superior para "Buscar Transacciones", "Retiro", "Depósito" e "Imprimir", facilitando la gestión de nuevas operaciones y la generación de reportes. Cada transacción tiene botones de acción para gestión individual, manteniendo la coherencia con el resto del sistema bancario.

14. Vista buscar transacciones



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

Inicio
Tarjetas
Clientes
Cuentas
Transacciones
Reportes
Oficinas
Cajero

Búsqueda de Transacciones

Cuenta: Todas las cuentas Tipo: Todos los tipos Fecha Inicio: dd/mm/aaaa Fecha Fin: dd/mm/aaaa

Buscar

Opciones avanzadas

Resultados de la Búsqueda

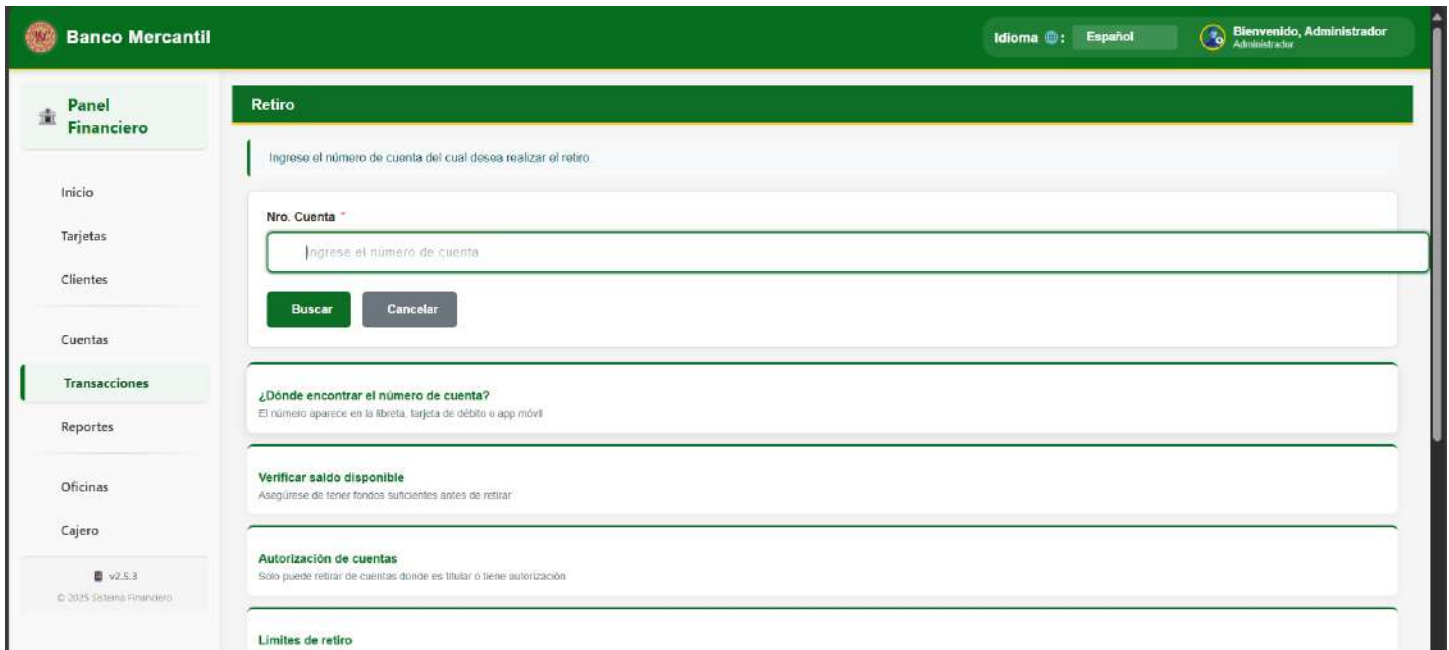
Fecha	Hora	Tipo	Nro. Cuenta	Cliente	Descripción	Monto	Acciones
15/01/2023	00:00:00	Otro	1234567904	Admin Sistema Banco	Retiro en cajero	\$ 450.00	 
14/01/2023	23:00:00	Otro	1234567903	Mario Sánchez Pérez	Depósito en ventanilla	\$ 800.00	 
13/01/2023	22:00:00	Otro	1234567902	Lucía Rodríguez Sánchez	Retiro en cajero	\$ 400.00	 
12/01/2023	21:00:00	Otro	1234567901	Jorge Martínez Rodríguez	Depósito en ventanilla	\$ 700.00	 
11/01/2023	20:00:00	Otro	1234567900	Carmen García Martínez	Retiro en cajero	\$ 350.00	 
10/01/2023	19:00:00	Otro	1234567899	Diego López García	Depósito en ventanilla	\$ 600.00	 

v2.5.3
© 2023 Sistema Financiero

La vista "Búsqueda de Transacciones" del Banco Mercantil proporciona una herramienta de filtrado avanzado para localizar operaciones específicas dentro del sistema. En la parte superior, ofrece filtros para seleccionar una Cuenta específica o "Todas las cuentas", Tipo de transacción o "Todos los tipos", y un rango de fechas con campos para Fecha Inicio y Fecha Fin en formato dd/mm/aaaa, todos acompañados de menús desplegables para facilitar la selección.

Un botón verde "Buscar" permite ejecutar la consulta, complementado con un enlace a "Opciones avanzadas" para filtros adicionales. En la sección "Resultados de la Búsqueda" se muestran las transacciones que coinciden con los criterios especificados, presentando información detallada de cada operación: Fecha, Hora, Tipo, Número de Cuenta, Cliente, Descripción, Monto y botones de Acciones.

15. Vista de transacción de retiro



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

- Inicio
- Tarjetas
- Clientes
- Cuentas
- Transacciones**
- Reportes
- Oficinas
- Cajero

Retiro

Ingrese el número de cuenta del cual desea realizar el retiro.

Nro. Cuenta *

Ingrese el número de cuenta

Buscar **Cancelar**

¿Dónde encontrar el número de cuenta?
El número aparece en la libreta, tarjeta de débito o app móvil

Verificar saldo disponible
Asegúrese de tener fondos suficientes antes de retirar

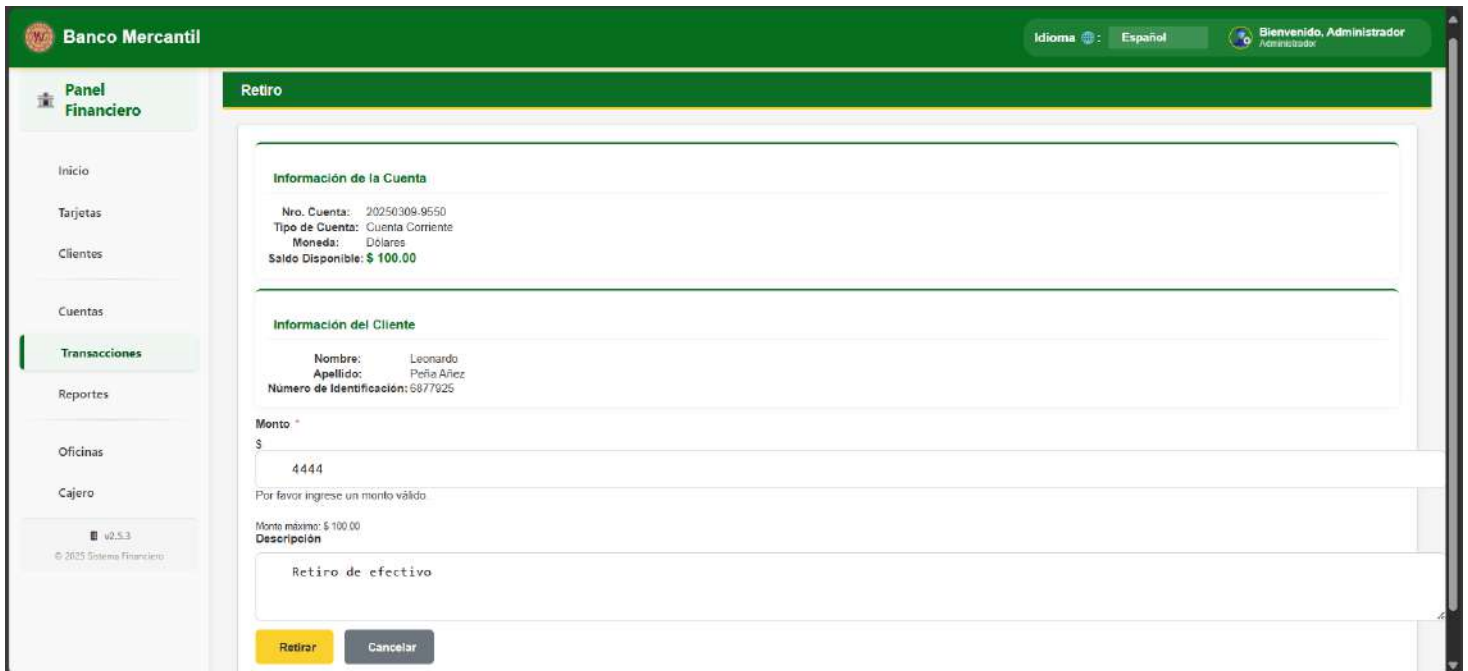
Autorización de cuentas
Solo puede retirar de cuentas donde es titular o tiene autorización

Límites de retiro

La vista "Retiro" del Banco Mercantil presenta una interfaz sencilla para iniciar operaciones de extracción de fondos. La pantalla solicita como primer paso el número de cuenta del cual se desea realizar el retiro mediante un campo de texto obligatorio, acompañado de botones "Buscar" y "Cancelar" para proceder o abandonar la operación.

La interfaz incluye secciones informativas que orientan al usuario durante el proceso: "¿Dónde encontrar el número de cuenta?" aclara que esta información aparece en la libreta, tarjeta de débito o app móvil. Adicionalmente, presenta recordatorios importantes sobre tres aspectos fundamentales de seguridad: "Verificar saldo disponible" (asegurándose de tener fondos suficientes), "Autorización de cuentas" (indicando que solo puede retirar de cuentas donde es titular o tiene autorización) y "Límites de retiro" (presumiblemente informando sobre restricciones de montos).

Esta vista inicial es el primer paso en el flujo de trabajo para efectuar retiros, manteniendo el diseño consistente con el resto del sistema bancario y proporcionando información contextual relevante para guiar al usuario.



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

- Inicio
- Tarjetas
- Cuentas
- Transacciones**
- Reportes
- Oficinas
- Cajero

Retiro

Información de la Cuenta

Nro. Cuenta: 20250309-9550
Tipo de Cuenta: Cuenta Corriente
Moneda: Dólares
Saldo Disponible: \$ 100.00

Información del Cliente

Nombre: Leonardo
Apellido: Peña Añez
Número de Identificación: 5877925

Monto: \$ 4444

Por favor ingrese un monto válido.

Monto máximo: \$ 100.00

Descripción: Retiro de efectivo

Retirar **Cancelar**

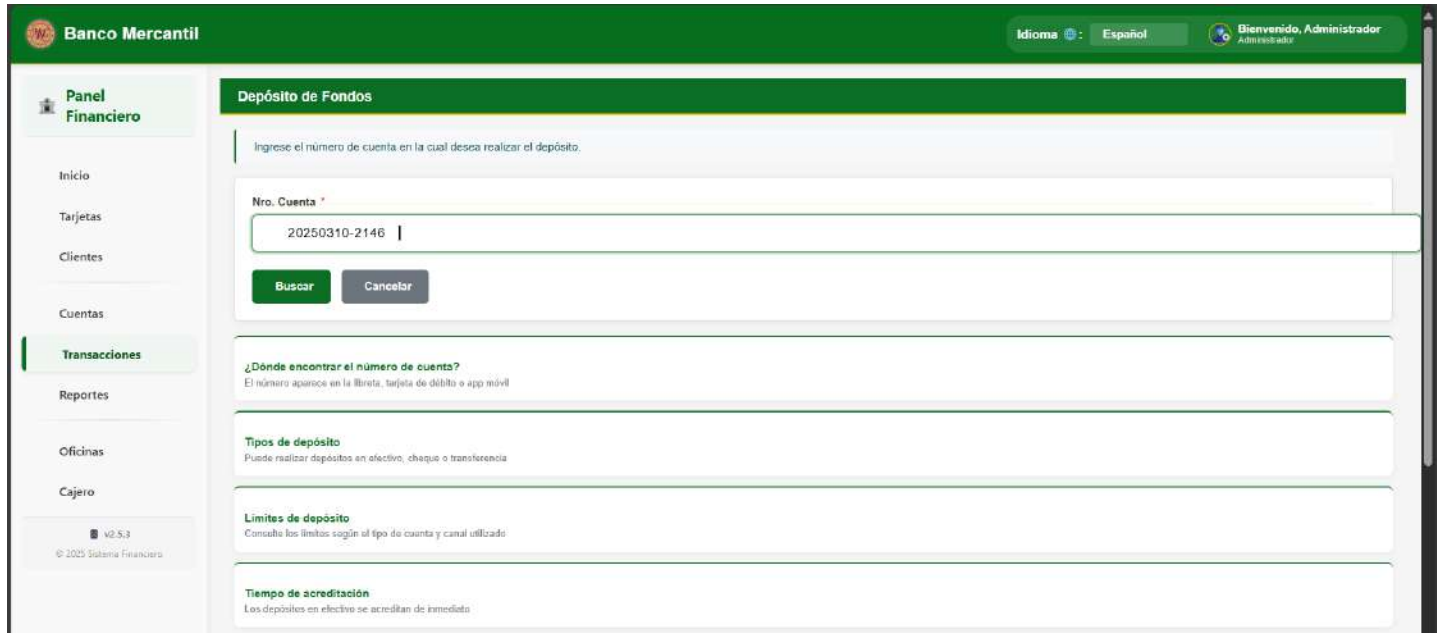
La segunda parte de la vista "Retiro" muestra el formulario completado con la información necesaria para procesar la extracción de fondos. La pantalla se divide en tres secciones principales:

En "Información de la Cuenta" se detallan los datos de la cuenta seleccionada: Número 20250309-8650, Tipo de Cuenta Corriente, Moneda Dólares y Saldo Disponible \$100.00.

La sección "Información del Cliente" identifica al titular: Leonardo Peña Añez, con Número de Identificación 5877925.

Finalmente, el área de transacción solicita el "Monto" a retirar (en este caso \$4444, aunque aparece un mensaje solicitando "Por favor ingrese un monto válido" y una nota indicando "Monto máximo: \$100.00"), seguido de un campo "Descripción" donde se ha ingresado "Retiro de efectivo". La interfaz concluye con botones para "Retirar" o "Cancelar" la operación.

16. Vista de transacción de Deposito



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

- Inicio
- Tarjetas
- Clientes
- Cuentas
- Transacciones**
- Reportes
- Oficinas
- Cajero

Depósito de Fondos

Ingrese el número de cuenta en la cual desea realizar el depósito.

Nro. Cuenta *

20250310-2146

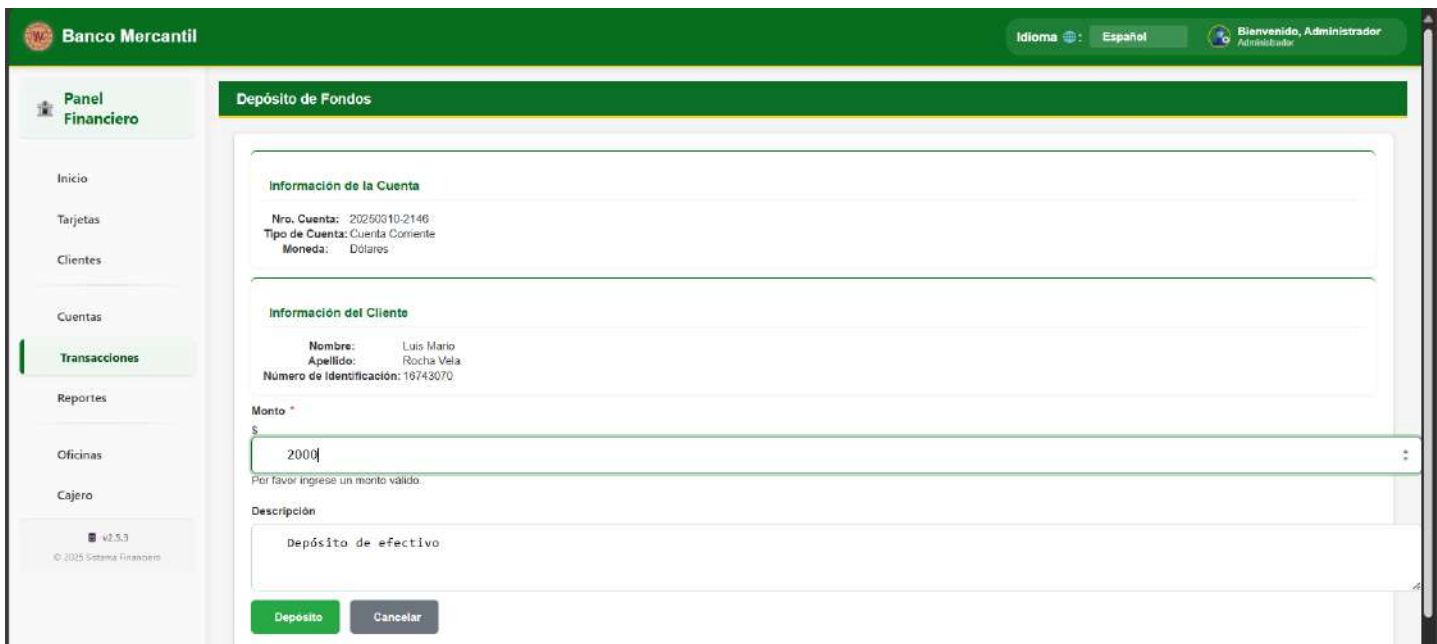
Buscar Cancelar

¿Dónde encontrar el número de cuenta?
El número aparece en la libreta, tarjeta de débito o app móvil

Tipos de depósito
Puede realizar depósitos en efectivo, cheque o transferencia

Limites de depósito
Consulte los límites según el tipo de cuenta y canal utilizado

Tiempo de acreditación
Los depósitos en efectivo se acreditan de inmediato



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

- Inicio
- Tarjetas
- Clientes
- Cuentas
- Transacciones**
- Reportes
- Oficinas
- Cajero

Depósito de Fondos

Información de la Cuenta

Nro. Cuenta: 20250310-2146
Tipo de Cuenta: Cuenta Corriente
Moneda: Dólares

Información del Cliente

Nombre: Luis Mario
Apellido: Rocha Vela
Número de identificación: 16743070

Monto *

\$ 2000

Por favor ingrese un monto válido.

Descripción

Depósito de efectivo

Deposito Cancelar

En esta sección no se entrara mucho en detalles porque hay las mismas vistas con las mismas funcionalidades que el tipo de transacción anterior, así que en pocas palabras tiene la misma metodología de trabajo pero en transacción Deposito.

17. Vista de ver detalles de transacción



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

- Inicio
- Tarjetas
- Cientes
- Cuentas
- Transacciones**
- Reportes
- Oficinas
- Cajero

Detalles de Transacción Modo Visualización

Transacción #13 Otra Operación + \$ 111,111.00 10/03/2025 a las 21:24:20

Información de la Transacción

ID Transacción	Tipo
13	Otro
Fecha	Hora
10/03/2025	21:24:20
Monto	Saldo Resultante
\$ 111,111.00	\$ 111,111.00
Descripción	
Cuenta Origen	
1	
Cuenta Destino	
1	

Información de la Cuenta

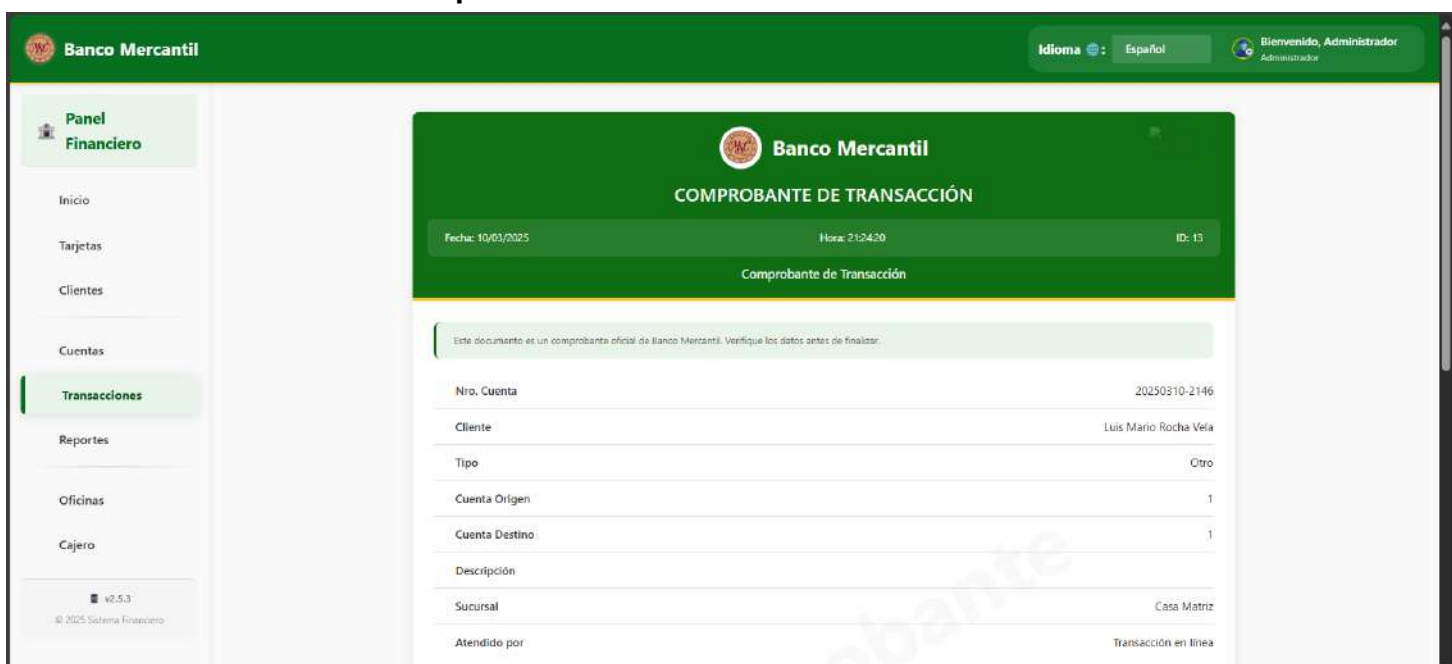
Nro. Cuenta	Tipo de Cuenta
20250310-2146	Corriente
Moneda	
Dólares	

Información del Cliente

Nombre Completo
Luis Mario Rocha Vela
CI / Documento
16743070

La vista "Detalles de Transacción" muestra información sobre la Transacción #13, un depósito de \$111,111.00 realizado el 10/03/2025. La pantalla organiza los datos en tres secciones: detalles de la transacción (ID, fecha, hora, monto), información de la cuenta (número 20250310-2146, tipo Corriente) y datos del cliente (Luis Mario Rocha Vela). Esta interfaz facilita el seguimiento y verificación de las operaciones bancarias, manteniendo el diseño consistente con el resto del sistema.

18. Vista ver Comprobante



Banco Mercantil Idioma: Español Bienvenido, Administrador

Panel Financiero

- Inicio
- Tarjetas
- Cientes
- Cuentas
- Transacciones**
- Reportes
- Oficinas
- Cajero

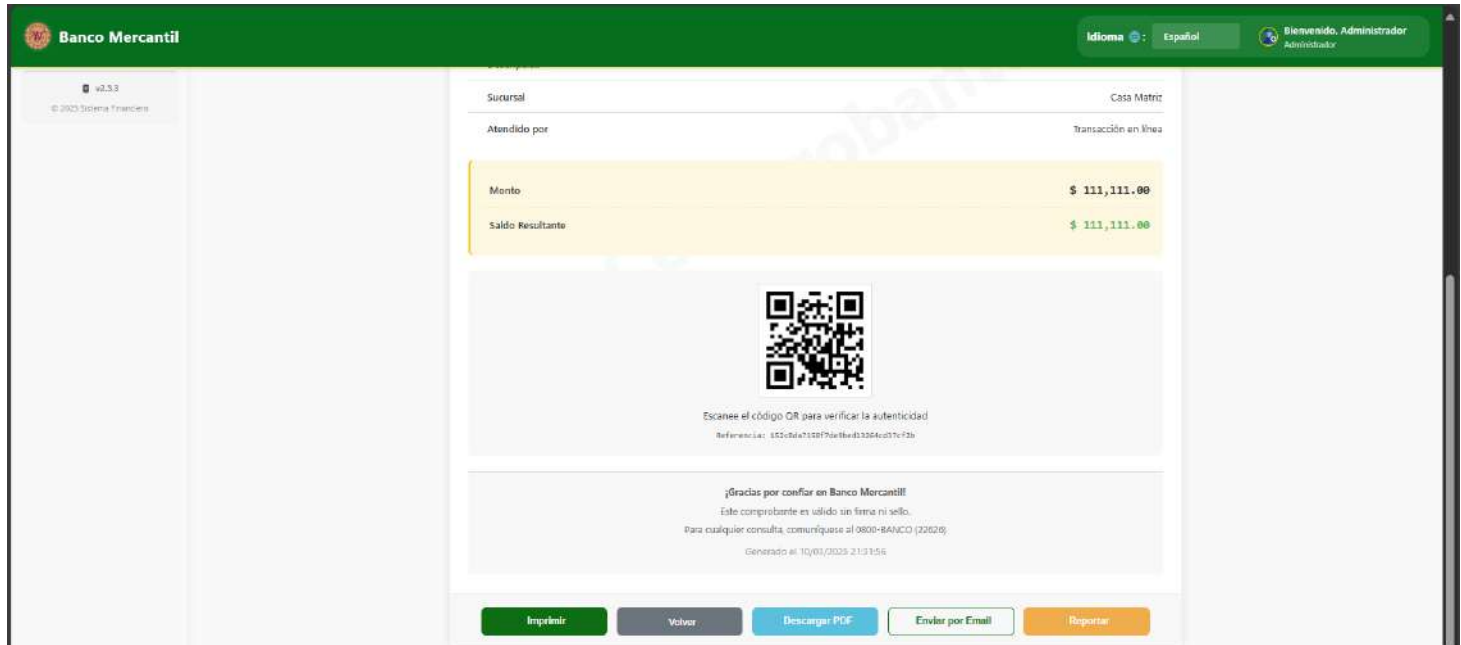
COMPROBANTE DE TRANSACCIÓN

Fecha: 10/03/2025 Hora: 21:24:20 ID: 13

Comprobante de Transacción

Este documento es un comprobante oficial de Banco Mercantil. Verifique los datos antes de finalizar.

Nro. Cuenta	20250310-2146
Cliente	Luis Mario Rocha Vela
Tipo	Otro
Cuenta Origen	1
Cuenta Destino	1
Descripción	
Sucursal	Casa Matriz
Atendido por	Transacción en línea



La vista "Comprobante de Transacción" presenta un documento oficial que certifica una operación financiera realizada en el Banco Mercantil. El comprobante muestra los detalles de la Transacción ID 13, realizada el 10/03/2025 a las 21:24:20 en la cuenta 20250310-2146 perteneciente a Luis Mario Rocha Vela.

El documento detalla una transacción tipo "Otro" entre Cuenta Origen 1 y Cuenta Destino 1, procesada en la sucursal Casa Matriz como "Transacción en línea". Destaca el monto de \$111,111.00 y el saldo resultante por el mismo valor, presentados en un recuadro amarillo.

Para verificar la autenticidad, incluye un código QR con un número de referencia y la aclaración "Este comprobante es válido sin firma ni sello". Al pie del documento aparecen cinco botones de acción: Imprimir, Volver, Descargar PDF, Enviar por Email y Reportar, junto con un número de contacto (0800-BANCO) para consultas y la fecha y hora de generación del comprobante.

19. Vista ver Reportes



Reportes

Nota: Los reportes se generan en tiempo real con datos actualizados del sistema.

Reportes de Transacciones y Estadísticas

Nombre del Reporte	Descripción	Acciones
Transacciones por Fecha	Informe detallado de transacciones realizadas en un rango de fechas seleccionado.	[Botón]
Transacciones por Cliente	Historial de transacciones agrupadas por cliente para un análisis detallado de actividad.	[Botón]
Estadísticas de ATM	Análisis de uso de cajeros automáticos, transacciones y disponibilidad.	[Botón]

Reportes de Saldos y Actividad

Nombre del Reporte	Descripción	Acciones
Saldo por Oficina	Reporte de saldos consolidados por oficina y tipo de cuenta.	[Botón]
Actividad de Cuenta	Historial detallado de actividades para todas las cuentas en un periodo seleccionado.	[Botón]
Resumen Ejecutivo	Resumen general de estadísticas e indicadores clave para la gerencia.	[Botón]

Reportes de Rendimiento y Auditoría

Nombre del Reporte	Descripción	Acciones
Rendimiento por Producto	Análisis de rendimiento para cada producto financiero ofrecido por el banco.	[Botón]
Auditoría del Sistema	Registro de todas las operaciones y cambios realizados en el sistema por usuario.	[Botón]
Métricas de Servicio	Estadísticas de calidad de servicio y satisfacción del cliente en todas las sucursales.	[Botón]

La vista "Reportes" del Banco Mercantil ofrece acceso a nueve tipos de informes organizados en tres categorías: Transacciones y Estadísticas (seguimiento de operaciones por fecha, cliente y uso de ATMs), Saldos y Actividad (consolidados por oficina, historial de cuentas y resumen ejecutivo), y Rendimiento y Auditoría (análisis de productos, registro de operaciones y métricas de servicio). Cada opción incluye una breve descripción de su función y un botón de acción para su generación. La interfaz destaca que todos los reportes se generan con datos en tiempo real, y ofrece la opción de imprimirlos mediante un botón destacado en amarillo.

20. Vista Listar Oficinas



Lista de Oficinas

Nueva Oficina

ID	Nombre	Dirección	Ciudad	Teléfono	Tipo	Estado	Acciones
10	Agencia Centro	Av. Cañoto #150	Santa Cruz de la Sierra	3-3456794	Sucursal	Inactiva	[Botón]
17	Agencia Cotoca	Av. Cotoca #700	Cotoca	3-3456801	Sucursal	Inactiva	[Botón]
18	Agencia El Torno	Av. El Torno #800	El Torno	3-3456802	Sucursal	Inactiva	[Botón]
12	Agencia Equipetrol	Av. San Martín #250	Santa Cruz de la Sierra	3-3456796	Sucursal	Inactiva	[Botón]
4	Agencia Este	Av. Virgen de Cotoca #300, Santa Cruz de la Sierra	Santa Cruz de la Sierra	3-3456792	Sucursal	Activa	[Botón]
8	Agencia Este	Av. Virgen de Cotoca #300	Santa Cruz de la Sierra	3-3456792	Sucursal	Inactiva	[Botón]
14	Agencia La Guardia	Av. La Guardia #450	Santa Cruz de la Sierra	3-3456796	Sucursal	Activa	[Botón]
16	Agencia Montero	Av. Montero #600	Montero	3-3456800	Sucursal	Activa	[Botón]
2	Agencia Norte	Av. Banzer Km 5, Santa Cruz de la Sierra	Santa Cruz de la Sierra	3-3456790	Sucursal	Activa	[Botón]
6	Agencia Norte	Av. Banzer Km 5	Santa Cruz de la Sierra	3-3456790	Sucursal	Activa	[Botón]
9	Agencia Oeste	Av. Cristo Redentor #200	Santa Cruz de la Sierra	3-3456793	Sucursal	Activa	[Botón]
11	Agencia Piraí	Av. Piraí #400	Santa Cruz de la Sierra	3-3456795	Sucursal	Activa	[Botón]

La vista "Lista de Oficinas" del Banco Mercantil presenta una tabla organizada con todas las sucursales de la entidad financiera. La interfaz muestra información clave de cada agencia: ID, Nombre, Dirección, Ciudad, Teléfono, Tipo (todas clasificadas como "Sucursal") y Estado (Activa o Inactiva).

Las oficinas se distribuyen principalmente en Santa Cruz de la Sierra, con algunas ubicadas en localidades como Cotoca, El Torno y Montero. Cada registro incluye botones de acción representados por íconos de colores para gestionar la información de las sucursales. Un botón amarillo "Nueva Oficina" en la parte superior permite agregar nuevas locaciones al sistema, manteniendo la coherencia visual con otras interfaces del Panel Financiero.

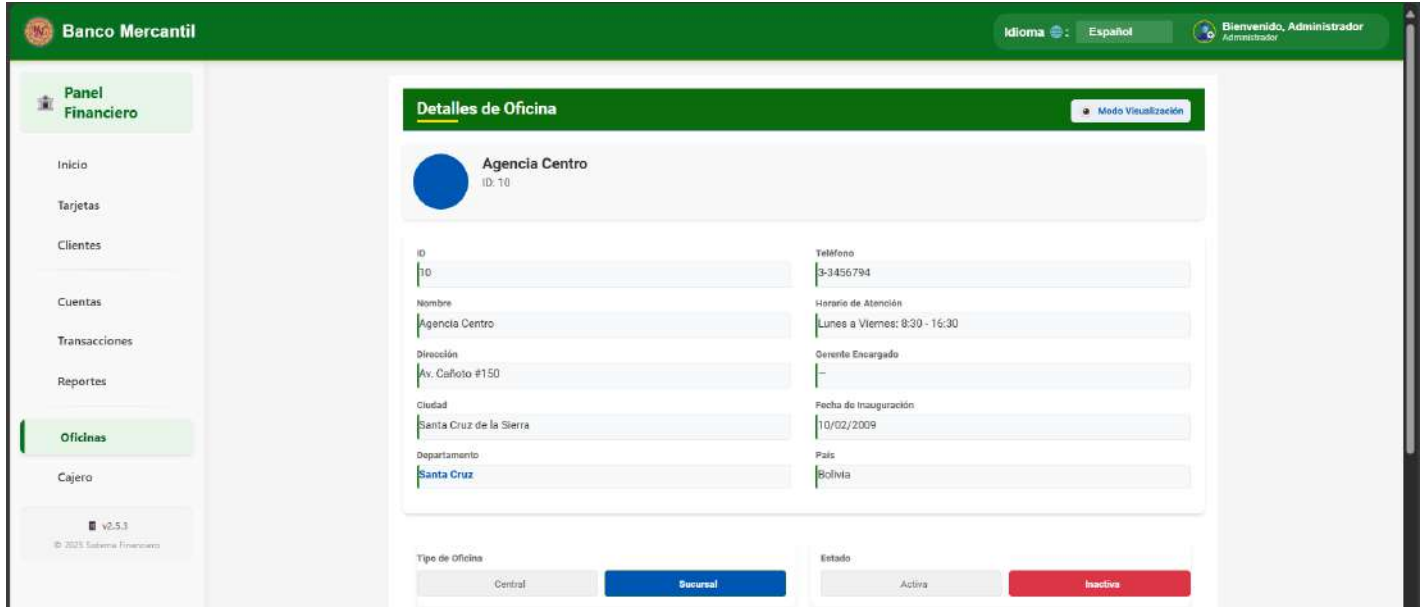
21. Vista de registrar Oficina



La vista "Nueva Oficina" del Banco Mercantil presenta un formulario para registrar sucursales en el sistema. La interfaz solicita información esencial como Nombre y Dirección (campos obligatorios marcados con asterisco), Teléfono de contacto y Tipo de Oficina (preseleccionado como "Sucursal").

El formulario incluye campos de ubicación geográfica: Ciudad (Santa Cruz de la Sierra), Departamento (Santa Cruz) y País (Bolivia), todos con valores predeterminados. También requiere especificar el Estado inicial (preconfigurado como "Activa") y el Horario de Atención, que muestra un ejemplo estándar: "Lunes a Viernes: 8:30 - 16:30, Sábados: 8:30 - 12:30".

22. Vista ver detalles de oficina

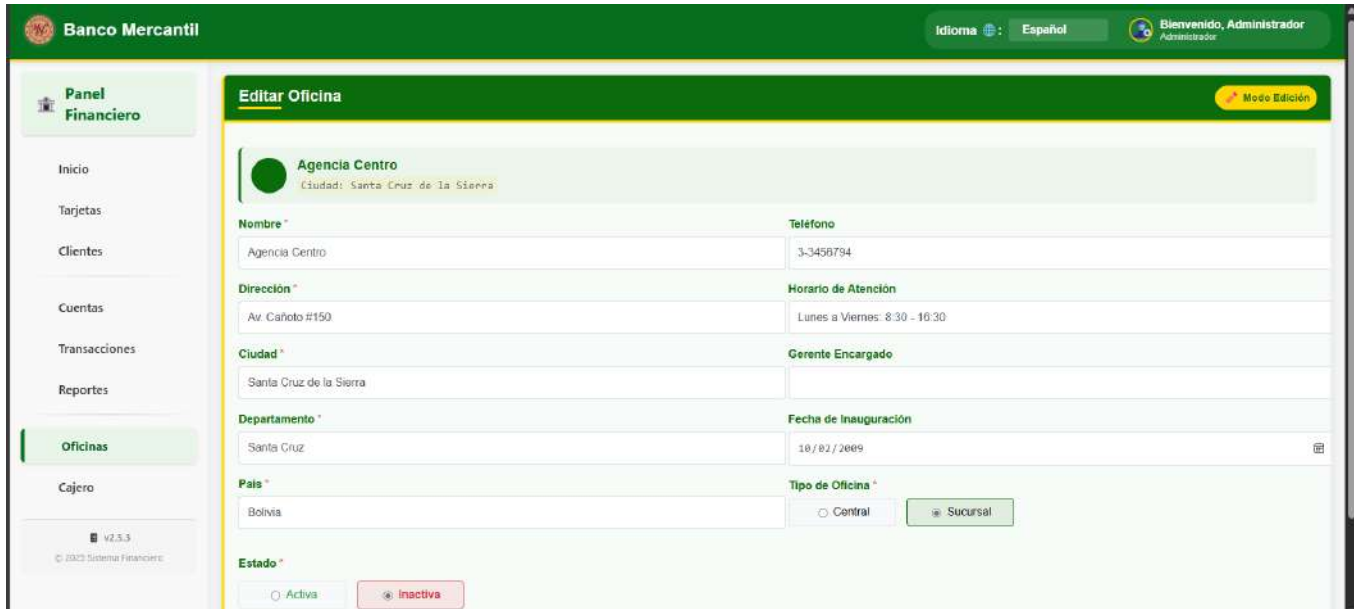


Detalles de Oficina	
Agencia Centro ID: 10	
ID	Teléfono
10	3-3456794
Nombre	Horario de Atención
Agencia Centro	Lunes a Viernes: 8:30 - 16:30
Dirección	Gerente Encargado
Av. Cañoto #150	
Ciudad	Fecha de Inauguración
Santa Cruz de la Sierra	10/02/2009
Departamento	País
Santa Cruz	Bolivia
Tipo de Oficina	
Central	Sucursal
Estado	
Activa	Inactiva

La vista "Detalles de Oficina" del Banco Mercantil muestra información completa sobre la Agencia Centro (ID: 10). La pantalla presenta los datos organizados en dos columnas: a la izquierda se muestran ID, Nombre, Dirección (Av. Cañoto #150), Ciudad (Santa Cruz de la Sierra) y Departamento (Santa Cruz); a la derecha aparecen Teléfono (3-3456794), Horario de Atención (Lunes a Viernes 8:30-16:30), Gerente Encargado, Fecha de Inauguración (10/02/2009) y País (Bolivia).

En la parte inferior se visualizan las opciones de Tipo de Oficina (con "Sucursal" seleccionada en azul) y Estado (con opciones para cambiar entre Activa e Inactiva). La interfaz cuenta con un botón "Modo Visualización" en la esquina superior derecha que permite alternar entre diferentes formas de ver la información, manteniendo la coherencia con otras pantallas del sistema.

23. Vista editar Oficina



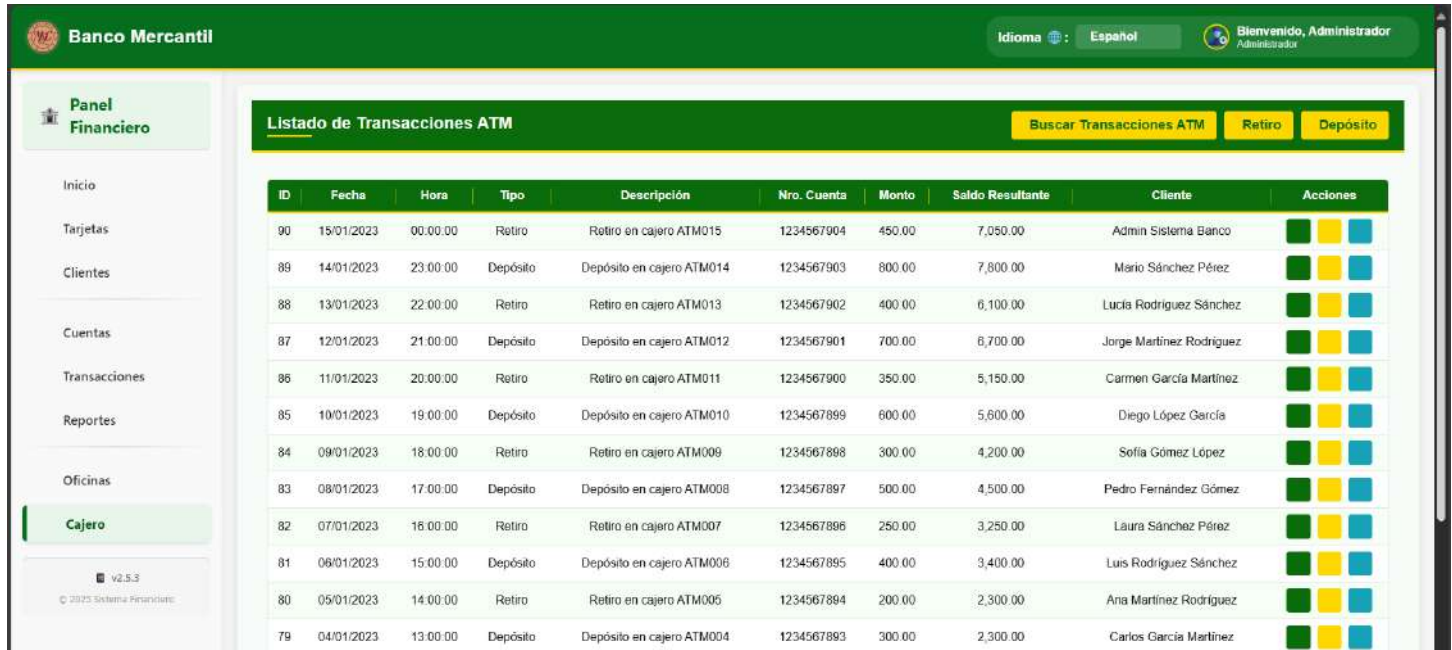
The screenshot shows the 'Editar Oficina' (Edit Office) form in the Banco Mercantil system. The form is for 'Agencia Centro' and contains the following fields:

- Nombre ***: Agencia Centro
- Teléfono**: 3-3456794
- Dirección ***: Av. Cañoto #150
- Horario de Atención**: Lunes a Viernes: 8:30 - 16:30
- Ciudad ***: Santa Cruz de la Sierra
- Gerente Encargado**
- Departamento ***: Santa Cruz
- Fecha de Inauguración**: 18/02/2009
- País ***: Bolivia
- Tipo de Oficina ***: ☐ Central ☒ Sucursal
- Estado ***: ☐ Activa ☒ Inactiva

The form is organized into two columns. The left column contains fields for Name, Address, City, Department, and Country. The right column contains fields for Phone, Hours, Manager, Opening Date, Office Type, and Status. The status is currently set to 'Inactiva'.

La vista "Editar Oficina" muestra un formulario para modificar los datos de la Agencia Centro. La interfaz organiza la información en dos columnas con campos editables que incluyen Nombre, Dirección, Ciudad, Departamento y País en el lado izquierdo; Teléfono, Horario de Atención, Gerente Encargado, Fecha de Inauguración y Tipo de Oficina en el derecho. En la parte inferior se puede cambiar el Estado entre Activa e Inactiva mediante botones de selección. Los campos obligatorios están marcados con asterisco, y la pantalla mantiene el diseño consistente con el resto del sistema bancario.

24. Vista listar transacciones ATM



ID	Fecha	Hora	Tipo	Descripción	Nro. Cuenta	Monto	Saldo Resultante	Cliente	Acciones
90	15/01/2023	00:00:00	Retiro	Retiro en cajero ATM015	1234567904	450.00	7,050.00	Admin Sistema Banco	  
89	14/01/2023	23:00:00	Depósito	Depósito en cajero ATM014	1234567903	800.00	7,800.00	Mario Sánchez Pérez	  
88	13/01/2023	22:00:00	Retiro	Retiro en cajero ATM013	1234567902	400.00	6,100.00	Lucía Rodríguez Sánchez	  
87	12/01/2023	21:00:00	Depósito	Depósito en cajero ATM012	1234567901	700.00	6,700.00	Jorge Martínez Rodríguez	  
86	11/01/2023	20:00:00	Retiro	Retiro en cajero ATM011	1234567900	350.00	5,150.00	Carmen García Martínez	  
85	10/01/2023	19:00:00	Depósito	Depósito en cajero ATM010	1234567899	600.00	5,600.00	Diego López García	  
84	09/01/2023	18:00:00	Retiro	Retiro en cajero ATM009	1234567898	300.00	4,200.00	Sofía Gómez López	  
83	08/01/2023	17:00:00	Depósito	Depósito en cajero ATM008	1234567897	500.00	4,500.00	Pedro Fernández Gómez	  
82	07/01/2023	16:00:00	Retiro	Retiro en cajero ATM007	1234567896	250.00	3,250.00	Laura Sánchez Pérez	  
81	06/01/2023	15:00:00	Depósito	Depósito en cajero ATM006	1234567895	400.00	3,400.00	Luis Rodríguez Sánchez	  
80	05/01/2023	14:00:00	Retiro	Retiro en cajero ATM005	1234567894	200.00	2,300.00	Ana Martínez Rodríguez	  
79	04/01/2023	13:00:00	Depósito	Depósito en cajero ATM004	1234567893	300.00	2,300.00	Carlos García Martínez	  

La vista "Listado de Transacciones ATM" del Banco Mercantil muestra un registro cronológico de operaciones realizadas en cajeros automáticos. La interfaz presenta una tabla con información detallada de cada transacción: ID, Fecha, Hora, Tipo (Retiro o Depósito), Descripción (con identificador específico del cajero), Número de Cuenta, Monto, Saldo Resultante y Cliente.

Los registros aparecen ordenados por fecha, del más reciente al más antiguo, alternando entre operaciones de retiro y depósito realizadas por diferentes clientes. En la parte superior, tres botones permiten "Buscar Transacciones ATM", realizar un nuevo "Retiro" o "Depósito". Cada transacción incluye botones de acción para su gestión individual, manteniendo la coherencia visual con el resto del sistema bancario.

CONCLUSIONES DEL PROYECTO

El proyecto "Implementación de un sistema web para la gestión bancaria con operaciones financieras" ha sido una experiencia integral que abarcó aspectos técnicos y académicos, cumpliendo con los objetivos planteados inicialmente. A continuación, se resumen las principales conclusiones:

- **Aplicación de Buenas Prácticas en Desarrollo de Software:** Se aplicaron buenas prácticas de modelado y arquitectura de sistemas mediante UML, utilizando diagramas de clases, casos de uso, secuencia y actividad. Este enfoque estructurado facilitó la comprensión y ejecución del sistema.
- **Impacto Académico y Reutilización:** El proyecto representó una valiosa oportunidad académica para aplicar conocimientos en software, arquitectura de sistemas y seguridad informática. A diferencia de las soluciones comerciales cerradas, el sistema está bien documentado, lo que lo convierte en una herramienta útil para estudiantes y desarrolladores interesados en integrar sistemas bancarios.
- **Optimización de la Gestión de Transacciones:** Se implementó un sistema seguro y eficiente para la gestión de transacciones (depósitos, retiros, transferencias) mediante un módulo API que permitió la integración en tiempo real con los cajeros automáticos (ATM), mejorando la disponibilidad y confiabilidad de los servicios bancarios.
- **Aprendizaje del Área Bancaria:** El proyecto proporcionó una comprensión profunda de los sistemas bancarios digitales, especialmente en la gestión de transacciones y la integración con cajeros automáticos. Se destacó la importancia de la seguridad, eficiencia y escalabilidad en el desarrollo de plataformas bancarias.
- **Importancia del Trabajo Grupal:** La colaboración efectiva entre los miembros del equipo fue crucial para el éxito del proyecto. Cada integrante contribuyó con sus habilidades, lo que facilitó el desarrollo del sistema. La organización, la distribución de tareas y la comunicación constante fueron claves para alcanzar los objetivos.

En pocas palabras, el proyecto cumplió sus objetivos técnicos y académicos, demostrando la importancia de la planificación estratégica, la implementación de las buenas prácticas de desarrollo generó el mejor resultado posible. El éxito del proyecto evidencia el potencial de los sistemas abiertos y documentados para fomentar el aprendizaje y mejorar los procesos en el ámbito financiero.

RECOMENDACIONES GRUPALES

Durante el desarrollo de este proyecto, hemos adquirido valiosas lecciones que pueden servir como guía para aquellos que deseen iniciarse en el área de desarrollo o replicar proyectos similares. A continuación, compartimos nuestras recomendaciones basadas en nuestros aciertos y errores:

- **Planificación Estratégica:** Es esencial establecer objetivos claros, asignar tareas de manera equitativa y fijar un cronograma realista. Esto ayuda a gestionar el tiempo y los recursos de forma eficiente, reduciendo posibles retrasos y asegurando que cada fase del proyecto se complete con éxito.
- **Dominio de Herramientas Tecnológicas:** Es importante tener un buen manejo de herramientas como HTML y CSS. Aunque no utilizamos frameworks, conocer plataformas como Bootstrap y Angular puede ser muy útil. Practicar con ejemplos sencillos acelera el aprendizaje y mejora la calidad del trabajo, evitando errores en la implementación.
- **Pruebas y Optimización:** Realizar pruebas de forma regular y mejorar el código, la velocidad de carga y la compatibilidad con diversos dispositivos es clave para ofrecer una experiencia de usuario fluida, permitiendo detectar problemas antes de que se conviertan en dificultades más grandes.
- **Documentación Detallada:** Mantener una documentación exhaustiva sobre decisiones tomadas, configuraciones y cambios realizados facilita la solución de problemas y permite la integración de nuevos desarrolladores, mejorando la capacidad de mantenimiento y futuras actualizaciones del sistema.
- **Modelado UML:** Aunque al principio puede parecer innecesario o complicado, el modelado UML (estructural y de comportamiento) es vital para orientar el desarrollo del proyecto. Los diagramas proporcionan claridad y coherencia entre los diferentes módulos, evitando errores y facilitando la resolución de problemas a largo plazo.
- **Iteración y Retroalimentación:** Adoptar un enfoque flexible y ajustar el proyecto según la retroalimentación recibida permite identificar áreas de mejora y hacer ajustes antes de la finalización, asegurando que el producto cumpla con las expectativas y requisitos establecidos.

Al aplicar estas recomendaciones, el proceso de desarrollo será más ágil, eficiente y exitoso en futuros sistemas web u otros proyectos de desarrollo.

REFERENCIAS BIBLIOGRAFICAS

- Argenis. (2024, Abril 05). *webempresa*. Obtenido de PHP, ¿Qué es y cómo funciona?:
<https://www.webempresa.com/blog/php-que-es-y-como-funciona.html>
- Code, L. (2022, Junio 22). *dev.to*. Obtenido de La Cascada en CSS: Qué es y cómo funciona:
<https://dev.to/lupitacode/la-cascada-en-css-que-es-y-como-funciona-31cd>
- Cuadrado, G. C. (2022, Julio 22). *openwebinars*. Obtenido de Visual Studio Code: Editor de código para desarrolladores: <https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/>
- Cyberstream. (2024, Mayo 30). *byronvargas*. Obtenido de XAMPP: Guía completa sobre qué es y para qué sirve: <https://www.byronvargas.com/web/que-es-y-para-que-sirve-el-xampp/>
- Delgado, H. (2024, Abril 25). *disenowebakus*. Obtenido de Historia de HTML - Origen y evolución del hipertexto Web: <https://disenowebakus.net/historia-html.php>
- Luna, J. C. (2024, Septiembre 11). *datacamp*. Obtenido de Tutorial de MySQL: Guía completa para principiantes: <https://www.datacamp.com/es/tutorial/my-sql-tutorial>
- Maldonado, J. (2021, Junio 01). *joystick*. Obtenido de Historia de PHP sus ventajas, desventajas y para que se usa actualmente.: <https://www.joystick.com.mx/historia-de-php-y-sus-ventajas-y-desventajas-y-para-que-se-usa-actualmente/>
- Peña, V. (2022, Julio 19). *norvicsoftware*. Obtenido de ¿Qué es Xampp?:
<https://norvicsoftware.com/que-es-xampp/>
- recluit. (2022, Abril 12). *recluit.com*. Obtenido de ¿Qué es Visual Studio Code?:
<https://recluit.com/que-es-visual-studio-code/>
- Robledano, A. (2019, Diciembre 24). *openwebinars*. Obtenido de Qué es MySQL: Características y ventajas: <https://openwebinars.net/blog/que-es-mysql/>
- Robledano, A. (2020, Mayo 14). *openwebinars*. Obtenido de Qué es Github:
<https://openwebinars.net/blog/que-es-github/>
- Saavedra, J. A. (2023, Junio 04). *ebac*. Obtenido de Qué es Github y para qué sirve: una guía para principiantes: <https://ebac.mx/blog/que-es-github>
- vadavo. (2024, Noviembre 18). *vadavo.com*. Obtenido de Conceptos básicos de HTML: Guía para principiantes: <https://www.vadavo.com/blog/html-que-es-y-para-que-sirve/>
- Zermeño, R. M. (2022, Agosto 26). *azulschool.net*. Obtenido de CSS, historia de los estilos y el diseño web.: <https://azulschool.net/css-historia-de-los-estilos-y-el-diseno-web/>

ANEXOS

```
-- -----  
-- Script completo para la base de datos 'banco_camba'  
-- -----  
  
-- Crear la base de datos  
CREATE DATABASE banco_camba;  
USE banco_camba;  
  
-- -----  
-- Estructura de la base de datos  
-- -----  
  
-- Tabla Oficina (ampliada con más campos para el formulario)  
CREATE TABLE oficina (  
    idOficina INT not null AUTO_INCREMENT,  
    hash varchar(50) COMMENT 'hash',  
    central BOOLEAN NOT NULL,  
    nombre VARCHAR(100) NOT NULL,  
    direccion VARCHAR(255) NOT NULL,  
    telefono VARCHAR(20),  
    ciudad VARCHAR(100) NOT NULL DEFAULT 'Santa Cruz de la Sierra',  
    departamento VARCHAR(100) NOT NULL DEFAULT 'Santa Cruz',  
    pais VARCHAR(100) NOT NULL DEFAULT 'Bolivia',  
    horarioAtencion VARCHAR(255),  
    gerenteEncargado VARCHAR(100),  
    fechaInauguracion DATE,  
    estado ENUM('activa', 'inactiva') NOT NULL DEFAULT 'activa',  
    CONSTRAINT PK_Oficina PRIMARY KEY (idOficina)  
) Engine = InnoDB Charset = utf8;  
  
-- Tabla Persona  
CREATE TABLE persona (  
    idPersona INT not null AUTO_INCREMENT,  
    hash varchar(50) COMMENT 'hash',  
    nombre VARCHAR(100) NOT NULL,  
    apellidoMaterno VARCHAR(100) NOT NULL,  
    apellidoPaterno VARCHAR(100) NOT NULL,  
    direccion VARCHAR(255),  
    telefono VARCHAR(20),  
    email VARCHAR(100),  
    fechaNacimiento DATE,
```

```
ci VARCHAR(20),
idOficina INT,
CONSTRAINT PK_Persona PRIMARY KEY (idPersona),
CONSTRAINT FK_Persona FOREIGN KEY (idOficina) REFERENCES oficina
(idOficina)
) Engine = InnoDB Charset = utf8;

-- Tabla Usuario (contiene FK a Persona)
CREATE TABLE usuario (
  idUsuario INT not null AUTO_INCREMENT,
  hash varchar(50) COMMENT 'hash',
  ultimoInicioSesion TIMESTAMP NULL DEFAULT NULL,
  intentosFallido INT DEFAULT 0,
  username VARCHAR(50) NOT NULL,
  password BLOB NOT NULL,
  idPersona INT NOT NULL,
  CONSTRAINT PK_Usuario PRIMARY KEY (idUsuario),
  CONSTRAINT FK_Usuario FOREIGN KEY (idPersona) REFERENCES persona
(idPersona)
) Engine = InnoDB Charset = utf8;

-- Tabla Cuenta (contiene FK a Persona)
CREATE TABLE cuenta (
  idCuenta INT not null AUTO_INCREMENT,
  hash varchar(50) COMMENT 'hash',
  tipoCuenta ENUM('cuentaAhorro', 'cuentaCorriente') NOT NULL,
  tipoMoneda ENUM('bolivianos', 'dolares') NOT NULL,
  fechaApertura DATE NOT NULL,
  estado ENUM('activa', 'inactiva') NOT NULL DEFAULT 'activa',
  nroCuenta VARCHAR(20) NOT NULL,
  saldo DECIMAL(15,2) NOT NULL DEFAULT 0.0,
  idPersona INT NOT NULL,
  CONSTRAINT PK_Cuenta PRIMARY KEY (idCuenta),
  CONSTRAINT FK_Cuenta FOREIGN KEY (idPersona) REFERENCES persona
(idPersona)
) Engine = InnoDB Charset = utf8;

-- Tabla Tarjeta (contiene FK a Cuenta)
CREATE TABLE tarjeta (
  idTarjeta INT not null AUTO_INCREMENT,
  hash varchar(50) COMMENT 'hash',
  estado ENUM('activa', 'inactiva') NOT NULL DEFAULT 'activa',
  tipoTarjeta ENUM('debito', 'credito') NOT NULL,
  nroTarjeta VARCHAR(20) NOT NULL,
  cvv VARCHAR(3) NOT NULL,
```

```
fechaExpiracion VARCHAR(7) NOT NULL,  
pin VARCHAR(4) NOT NULL,  
idCuenta INT NOT NULL,  
CONSTRAINT PK_Tarjeta PRIMARY KEY (idTarjeta),  
CONSTRAINT FK_Tarjeta FOREIGN KEY (idCuenta) REFERENCES cuenta  
(idCuenta)  
) Engine = InnoDB Charset = utf8;  
  
-- Tabla ATM (con campos necesarios para el registro)  
CREATE TABLE atm (  
    idATM INT not null AUTO_INCREMENT,  
    hash varchar(50) COMMENT 'hash',  
    codigo VARCHAR(20) NOT NULL,  
    marca VARCHAR(100),  
    modelo VARCHAR(100),  
    ubicacion VARCHAR(255) NOT NULL,  
    direccion VARCHAR(255) NOT NULL,  
    ciudad VARCHAR(100) NOT NULL DEFAULT 'Santa Cruz de la Sierra',  
    departamento VARCHAR(100) NOT NULL DEFAULT 'Santa Cruz',  
    tipoATM ENUM('interno', 'externo') NOT NULL DEFAULT 'interno',  
    estado ENUM('activo', 'inactivo', 'mantenimiento') NOT NULL DEFAULT  
'activo',  
    idOficina INT,  
    fechaInstalacion DATE,  
    CONSTRAINT PK_Atm PRIMARY KEY (idATM),  
    CONSTRAINT FK_ATM_Oficina FOREIGN KEY (idOficina) REFERENCES oficina  
(idOficina)  
) Engine = InnoDB Charset = utf8;  
  
-- Tabla Transaccion (mantiene los campos originales)  
CREATE TABLE transaccion (  
    idTransaccion INT not null AUTO_INCREMENT,  
    hash varchar(50) COMMENT 'hash',  
    tipoTransaccion ENUM('retiro', 'deposito') NOT NULL,  
    fecha DATE NOT NULL,  
    hora TIME NOT NULL,  
    descripcion VARCHAR(255),  
    monto DECIMAL(15,2) NOT NULL,  
    idCuenta INT NOT NULL,  
    cuentaOrigen varchar(50) DEFAULT NULL,  
    cuentaDestino varchar(50) DEFAULT NULL,  
    saldoResultante decimal(15,2) DEFAULT NULL,  
    CONSTRAINT PK_Transaccion PRIMARY KEY (idTransaccion),  
    CONSTRAINT FK_Transaccion FOREIGN KEY (idCuenta) REFERENCES cuenta  
(idCuenta)
```

```
) Engine = InnoDB Charset = utf8;

-- Tabla TransaccionATM (similar a transaccion pero con campo para ATM)
CREATE TABLE transaccion_atm (
  idTransaccionATM INT not null AUTO_INCREMENT,
  hash varchar(50) COMMENT 'hash',
  tipoTransaccion ENUM('retiro', 'deposito', 'consulta') NOT NULL,
  fecha DATE NOT NULL,
  hora TIME NOT NULL,
  descripcion VARCHAR(255),
  monto DECIMAL(15,2) NOT NULL,
  idCuenta INT NOT NULL,
  cuentaOrigen varchar(50) DEFAULT NULL,
  cuentaDestino varchar(50) DEFAULT NULL,
  saldoResultante decimal(15,2) DEFAULT NULL,
  idATM INT NOT NULL,
  idTarjeta INT NOT NULL,
  CONSTRAINT PK_TransaccionATM PRIMARY KEY (idTransaccionATM),
  CONSTRAINT FK_TransaccionATM_Cuenta FOREIGN KEY (idCuenta) REFERENCES
cuenta (idCuenta),
  CONSTRAINT FK_TransaccionATM_ATM FOREIGN KEY (idATM) REFERENCES atm
(idATM),
  CONSTRAINT FK_TransaccionATM_Tarjeta FOREIGN KEY (idTarjeta) REFERENCES
tarjeta (idTarjeta)
) Engine = InnoDB Charset = utf8;

CREATE TABLE token (
  idToken INT not null AUTO_INCREMENT,
  token VARCHAR(255) NOT NULL,
  fechaCreacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  fechaExpiracion TIMESTAMP NOT NULL,
  idTarjeta INT NOT NULL,
  CONSTRAINT PK_Token PRIMARY KEY (idToken),
  CONSTRAINT FK_Token FOREIGN KEY (idTarjeta) REFERENCES tarjeta
(idTarjeta)
) Engine = InnoDB Charset = utf8;
-- create index idx_token on token(token);
CREATE INDEX idx_token ON token(token);
```

```
<?php

/* ----- CLASE DATABASE ----- */
class Database {
    // Propiedades de la clase para la configuración de la base de datos
    private $host = 'localhost';        // Host de la base de datos
    private $db_name = 'banco_camba';   // Nombre de la base de datos
    private $username = 'root';         // Usuario de la base de datos
    private $password = '';             // Contraseña de la base de datos
    private $conn;                      // Objeto de conexión a la base de
    datos

    /**
     * Método para conectar a la base de datos
     * @return PDO Retorna un objeto de conexión PDO
     */
    public function connect() {
        // Inicializa la conexión como nula
        $this->conn = null;

        try {
            // Intenta establecer la conexión usando PDO
            $this->conn = new PDO(
                'mysql:host=' . $this->host . ';dbname=' . $this->db_name,
                $this->username, // Nombre de usuario
                $this->password  // Contraseña
            );

            // Configura el manejo de errores para lanzar excepciones
            $this->conn->setAttribute(PDO::ATTR_ERRMODE,
                PDO::ERRMODE_EXCEPTION);

            // Establece el conjunto de caracteres a UTF-8
            $this->conn->exec("SET NAMES utf8");
        } catch(PDOException $e) {
            // Captura y muestra errores de conexión
            echo 'Connection Error: ' . $e->getMessage();
        }

        // Retorna el objeto de conexión
        return $this->conn;
    }
}

?>
```

```
/* ----- HTACCESS ----- */

<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteBase /upds/banco-mercantil/

    # Asegurarse de no redirigir archivos o directorios existentes
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteCond %{REQUEST_FILENAME} !-d

    # Redirigir todas las solicitudes que no sean para archivos o directorios
    # existentes a index.php
    RewriteRule ^(.*)$ index.php?url=$1 [QSA,L]

    # Permitir explícitamente redirecciones a index.php con parámetros
    RewriteCond %{THE_REQUEST} ^[A-
Z]{3,}\s/+index\.php\?controller=([^\s]+)&action=([^\s]+) [NC]
    RewriteRule ^ %1/%2? [R=301,L]
</IfModule>
```

```
<?php
function dd(...$paran)
{
    echo "<pre>";
    var_dump(...$paran);
    echo "</pre>";
    die();
}

// Función para verificar si el usuario está autenticado correctamente
function isAuthenticated() {
    // Verificar que existan todas las variables de sesión necesarias
    if (!isset($_SESSION['user_id']) ||
        !isset($_SESSION['username']) ||
        !isset($_SESSION['is_logged_in']) ||
        !isset($_SESSION['login_time'])) {
        return false;
    }
}
```

```
// Verificar que is_logged_in sea true
if ($_SESSION['is_logged_in'] !== true) {
    return false;
}

// Verificar tiempo de expiración de la sesión (opcional)
// Por ejemplo, 2 horas = 7200 segundos
$sessionLifetime = 7200;
if (time() - $_SESSION['login_time'] > $sessionLifetime) {
    // La sesión ha expirado, limpiar variables
    session_unset();
    session_destroy();
    return false;
}

// Todo verificado correctamente
return true;
}

// Función para redireccionar de manera segura con ruta absoluta
function redirect($path) {
    $base_url = (isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] === 'on' ?
"https" : "http");
    $base_url .= "://" . $_SERVER['HTTP_HOST'];
    $base_url .= str_replace(basename($_SERVER['SCRIPT_NAME']), "",
$_SERVER['SCRIPT_NAME']);

    header("Location: " . rtrim($base_url, '/') . '/' . ltrim($path, '/'));
    exit;
}

/* ----- BANCO CAMBA - PUNTO DE ENTRADA PRINCIPAL ----- */

// Iniciar buffer de salida para evitar problemas con redirecciones
ob_start();

// Implementación de cabeceras de seguridad HTTP
header("Strict-Transport-Security: max-age=31536000; includeSubDomains;
preload");
header("X-XSS-Protection: 1; mode=block");
header("X-Content-Type-Options: nosniff");
header("X-Frame-Options: SAMEORIGIN");
header("Content-Security-Policy: default-src 'self'; script-src 'self'
https://cdn.jsdelivr.net; style-src 'self' 'unsafe-inline'");
header("Referrer-Policy: strict-origin-when-cross-origin");
```



```
// Configuración para ocultar errores en producción
error_reporting(0);

// Iniciar sesión
session_start();

// Incluir archivos de configuración y utilidades
require_once 'config/database.php';
require_once 'utils/Session.php';
require_once 'utils/Auth.php';
require_once 'utils/Autoloader.php';

// Cargar modelos
require_once 'models/TransaccionATM.php';
require_once 'models/Oficina.php';
require_once 'models/Cliente.php';
require_once 'models/Cuenta.php';
require_once 'models/Transaccion.php';
require_once 'models/ATM.php';
require_once 'models/Tarjeta.php';
require_once 'models/Usuario.php';
require_once 'models/Bienvenida.php';
require_once 'models/Token.php';

// MODIFICACIÓN: Asegurarse de que el idioma está cargado correctamente
// Inicializar la sesión
$session = new Session();

// Obtener el idioma actual, si viene un idioma por GET, actualizarlo
if (isset($_GET['lang']) && in_array($_GET['lang'], ['es', 'en', 'pt'])) {
    $session->setLanguage($_GET['lang']);
    $_SESSION['lang'] = $_GET['lang'];
}

// Obtener el código de idioma
$lang_code = $session->getLanguage();

// Verificar que el archivo de idioma existe
if (!file_exists("langs/{$lang_code}.php")) {
    $lang_code = 'es'; // Idioma por defecto si no existe el archivo
}

// Cargar el archivo de idioma
require_once "langs/{$lang_code}.php";
```

```
// Determinar controlador y acción por defecto
$controller = isset($_GET['controller']) ? $_GET['controller'] : 'usuario';
$action = isset($_GET['action']) ? $_GET['action'] : 'login';

// Si el usuario ya está autenticado y trata de acceder a login, redirigir
al dashboard
if (isAuthenticated() && $controller == 'usuario' && $action == 'login') {
    redirect('index.php?controller=bienvenida&action=index');
}

// Lista de acciones públicas que no requieren autenticación
$publicActions = ['login', 'autenticar', 'cambiarIdioma'];

// Verificar si el usuario ha iniciado sesión para acciones protegidas
if (!isAuthenticated() && !in_array($action, $publicActions) && $controller
!= 'api') {
    // Redirigir al login si no ha iniciado sesión y está intentando acceder
a páginas protegidas
    redirect('index.php?controller=usuario&action=login');
}

// Cargar el controlador apropiado
$controllerName = ucfirst($controller) . 'Controller';
$controllerFile = "controllers/{$controllerName}.php";

// Verificar si el archivo del controlador existe
if (file_exists($controllerFile)) {
    require_once $controllerFile;

    // Crear instancia del controlador
    $controllerInstance = new $controllerName();

    // Verificar si el método de acción existe
    if (method_exists($controllerInstance, $action)) {
        // Ejecutar la acción
        try {
            $controllerInstance->$action();
        } catch (Exception $e) {
            // Log de errores en archivo para producción
            error_log("Error en {$controllerName}::{action}: " . $e-
>getMessage());
        }

        // En entorno de desarrollo mostrar detalles completos
    }
}
```

```
        if ($_SERVER['SERVER_NAME'] == 'localhost' ||
strpos($_SERVER['SERVER_NAME'], '192.168.') === 0) {
            echo "<pre>";
            var_dump($e);
            echo "</pre>";
            echo "<pre>";
            var_dump($e->getLine());
            echo "</pre>";
            echo "<pre>";
            var_dump($e->getMessage());
            echo "</pre>";
            die();
        } else {
            // En producción mostrar un mensaje de error genérico
            include 'views/error.php';
            die("Ha ocurrido un error inesperado. Por favor contacte al
administrador.");
        }
    }
} else {
    // Acción no encontrada
    include 'views/error.php';
    die("La acción '$action' no existe en el controlador
'$controllerName'.");
}
} else {
    // Controlador no encontrado
    include 'views/error.php';
    die("El controlador '$controllerName' no existe.");
}

// Cerrar buffer de salida
ob_end_flush();
```

```
<?php

class ApiController
{
    public $data = [];
    public $conn;
    public function __construct()
    {
        header('Access-Control-Allow-Origin: *');
        header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE');
        header('Access-Control-Allow-Headers: Content-Type');
        header('Content-Type: application/json');
        $this->data = json_decode(file_get_contents('php://input'), true);
        $database = new Database();
        $db = $database->connect();
        $this->conn = $db;
    }
    public function login()
    {
        $numberCard = $this->data['numberCard'];
        $pin = $this->data['pin'];
        if (!$numberCard || !$pin) {
            return $this->error('Number card and pin are required');
        }

        $tarjeta = (new Tarjeta($this->conn))->login($numberCard, $pin);
        if ($tarjeta === false) {
            return $this->error('Number card or pin are incorrect');
        }

        $token = $tarjeta->NewToken();
        $usuario = $tarjeta->getUsuario();
        $this->success([
            'numberCard' => $numberCard,
            'token' => $token,
            'user' => $usuario
        ]);
    }
    public function retirarMoney()
    {
        $token = $this->validateToken();
        if (!$token) {
            return $this->error('Token is invalid', 401);
        }
    }
}
```

```
$account_id = $this->data['account_id'];
$amount = $this->data['amount'];
if (!$account_id || !$amount) {
    return $this->error('Account id and amount are required');
}
if ($amount < 1) {
    return $this->error('Amount must be greater than 0');
}
$cuentas = $token->getCuentas();
$cuenta = null;
foreach ($cuentas as $c) {
    if ($c->idCuenta == $account_id) {
        $cuenta = $c;
        break;
    }
}
if (!$cuenta) {
    return $this->error('Account not found');
}
$cuenta = new Cuenta($this->conn);
$cuenta->idCuenta = $account_id;
if (!$cuenta->obtenerUna()) {
    return $this->error('Account not found');
}
if ($cuenta->saldo < $amount) {
    return $this->error('Insufficient funds');
}
$model = new Transaccion($this->conn);
$model->realizarRetiro($cuenta->idCuenta, $amount, "Rertiro ATM");
$this->success(['message' => 'Withdrawal successful']);
}
public function accounts()
{
    $token = $this->validateToken();
    if (!$token) {
        return $this->error('Token is invalid', 401);
    }
    $cuentas = $token->getCuentas();
    $this->success($cuentas);
}
/**
 * @return Token|null
 */
protected function validateToken()
{

```

```
// vemos en la petición actual el token en authorization o X-Mi-Token
$token = null;
$allHeader=getallheaders();
if (isset($allHeader['Authorization'])) {
    $token = $allHeader['Authorization'];
} elseif (isset($allHeader['X-Mi-Token'])) {
    $token = $allHeader['X-Mi-Token'];
}
// eliminar Bearer
$token = str_replace('Bearer ', '', $token);
return Token::validate($token, $this->conn);
}

protected function success($data = [], $statusCode = 200)
{
    return self::json([
        'success' => true,
        'data' => $data
    ], $statusCode);
}

protected function error($data = [], $statusCode = 400)
{
    return self::json([
        'success' => false,
        'data' => $data
    ], $statusCode);
}

public static function json($data, $statusCode = 200, $headers =
['Content-Type' => 'application/json'])
{
    http_response_code($statusCode);
    foreach ($headers as $name => $value) {
        header("$name: $value");
    }
    echo json_encode($data);
    exit;
}
}
```

```
<?php
/**
 * Authentication Utility
 * Handles user authentication and authorization
 */
class Auth {
    /**
     * Check if user is logged in
     * @return boolean
     */
    public function isLoggedIn() {
        return isset($_SESSION['user_id']) && !empty($_SESSION['user_id']);
    }

    /**
     * Authenticate user
     * @param string $username
     * @param string $password
     * @return boolean|array Returns user data on success, false on failure
     */
    public function login($username, $password) {
        // Connect to database
        $database = new Database();
        $db = $database->connect();

        // Prepare query
        $query = "SELECT u.idUsuario, u.username, u.password,
u.intentosFallido,
                p.idPersona, p.nombre, p.apellidoPaterno,
p.apellidoMaterno, p.idOficina
                FROM Usuario u
                INNER JOIN Persona p ON u.idPersona = p.idPersona
                WHERE u.username = :username";

        $stmt = $db->prepare($query);
        $stmt->bindParam(':username', $username);
        $stmt->execute();

        if ($stmt->rowCount() > 0) {
            $row = $stmt->fetch(PDO::FETCH_ASSOC);

            // Verify password (assuming it's stored as a hash)
            if (password_verify($password, $row['password'])) {
                // Reset failed attempts
                $this->resetFailedAttempts($row['idUsuario']);
            }
        }
    }
}
```

```
// Update last login time
$this->updateLastLogin($row['idUsuario']);

// Return user data
return [
    'id' => $row['idUsuario'],
    'username' => $row['username'],
    'nombre' => $row['nombre'],
    'apellido' => $row['apellidoPaterno'] . ' ' .
$row['apellidoMaterno'],
    'idPersona' => $row['idPersona'],
    'idOficina' => $row['idOficina']
];
} else {
    // Increment failed attempts
    $this->incrementFailedAttempts($row['idUsuario']);
    return false;
}
}

return false;
}

/**
 * Update the last login timestamp
 * @param int $userId
 */
private function updateLastLogin($userId) {
    $database = new Database();
    $db = $database->connect();

    $query = "UPDATE Usuario SET ultimoInicioSesion = NOW() WHERE
idUsuario = :id";
    $stmt = $db->prepare($query);
    $stmt->bindParam(':id', $userId);
    $stmt->execute();
}

/**
 * Reset failed login attempts
 * @param int $userId
 */
private function resetFailedAttempts($userId) {
    $database = new Database();
```



```
$db = $database->connect();

$query = "UPDATE Usuario SET intentosFallido = 0 WHERE idUsuario =
:id";
$stmt = $db->prepare($query);
$stmt->bindParam(':id', $userId);
$stmt->execute();
}

/**
 * Increment failed login attempts
 * @param int $userId
 */
private function incrementFailedAttempts($userId) {
    $database = new Database();
    $db = $database->connect();

    $query = "UPDATE Usuario SET intentosFallido = intentosFallido + 1
WHERE idUsuario = :id";
    $stmt = $db->prepare($query);
    $stmt->bindParam(':id', $userId);
    $stmt->execute();
}

/**
 * Log user out
 */
public function logout() {
    // Unset all session variables
    $_SESSION = array();

    // Destroy the session
    session_destroy();
}
}
?>
```

```
<?php
/**
 * Clase Autoloader para carga automática de clases
 */
class Autoloader {
    /**
     * Registrar el autoloader
     */
    public static function register() {
        spl_autoload_register(array(__CLASS__, 'autoload'));
    }

    /**
     * Autoload de clases
     * @param string $className Nombre de la clase a cargar
     */
    public static function autoload($className) {
        // Definir directorios donde buscar clases
        $directories = array(
            'models/',
            'controllers/',
            'utils/',
            'views/',
            'libs/',
            'api/',
            'assets/',
            'config/',
            'langs/'
        );

        // Verificar cada directorio
        foreach ($directories as $directory) {
            $file = $directory . $className . '.php';

            // Si existe el archivo, incluirlo
            if (file_exists($file)) {
                require_once $file;
                return;
            }
        }
    }
}
?>
```

```
<?php
/**
 * Clase para manejar sesiones en el sistema
 */
class Session {
    /**
     * Constructor
     */
    public function __construct() {
        // Iniciar sesión si no está iniciada
        if (session_status() == PHP_SESSION_NONE) {
            session_start();
        }
    }

    /**
     * Establecer datos de usuario después del login
     * @param array $userData
     */
    public function setUserData($userData) {
        $_SESSION['user_id'] = $userData['id'];
        $_SESSION['username'] = $userData['username'];
        $_SESSION['nombre'] = $userData['nombre'];
        $_SESSION['apellido'] = $userData['apellido'];
        $_SESSION['idPersona'] = $userData['idPersona'];
        $_SESSION['idOficina'] = $userData['idOficina'];
        $_SESSION['is_logged_in'] = true;
        $_SESSION['login_time'] = time();
    }

    /**
     * Obtener datos de usuario desde la sesión
     * @return array
     */
    public function getUserData() {
        if (isset($_SESSION['user_id'])) {
            return [
                'id' => $_SESSION['user_id'],
                'username' => $_SESSION['username'],
                'nombre' => $_SESSION['nombre'],
                'apellido' => $_SESSION['apellido'],
                'idPersona' => $_SESSION['idPersona'],
                'idOficina' => $_SESSION['idOficina'],
                'is_logged_in' => $_SESSION['is_logged_in'],
                'login_time' => $_SESSION['login_time']
            ];
        }
    }
}
```

```
];
}

return null;
}

// Implementacion de protección contra CSRF
/**
 * Genera un token CSRF para proteger formularios
 * MODIFICACIÓN: Agregado método si no existía previamente
 * @return string El token generado
 */
public function generateCSRFToken() {
    $token = bin2hex(random_bytes(32));
    $_SESSION['csrf_token'] = $token;
    return $token;
}

/**
 * Valida un token CSRF
 * @param string $token El token a validar
 * @return bool True si el token es válido
 */
public function validateCSRFToken($token) {
    if (isset($_SESSION['csrf_token']) && $_SESSION['csrf_token'] ===
$token) {
        // Renovar token
        $this->generateCSRFToken();
        return true;
    }
    return false;
}

/**
 * Establecer un mensaje flash para mostrarlo en la próxima carga de
página
 * @param string $type Tipo de mensaje (success, error, info, warning)
 * @param string $message Contenido del mensaje
 */
public function setFlashMessage($type, $message) {
    $_SESSION['flash_message'] = [
        'type' => $type,
        'message' => $message
    ];
}
```

```
/**
 * Obtener mensaje flash y eliminarlo de la sesión
 * @return array|null
 */
public function getFlashMessage() {
    if (isset($_SESSION['flash_message'])) {
        $message = $_SESSION['flash_message'];
        unset($_SESSION['flash_message']);
        return $message;
    }

    return null;
}

/**
 * Establecer idioma actual
 * @param string $lang Código de idioma (es, en)
 * 02/03/2025: Asegurado que el idioma se guarde correctamente en la
sesión
 */
public function setLanguage($lang) {
    $_SESSION['lang'] = $lang;
}

/**
 * Obtener idioma actual
 * @return string
 * 02/03/2025: Añadida validación para asegurar que siempre retorne un
idioma válido
 */
public function getLanguage() {
    return isset($_SESSION['lang']) && in_array($_SESSION['lang'],
['es', 'en']) ? $_SESSION['lang'] : 'es';
}
?>
```